

Claude Code 完全実践ガイド

スキル・フック・メモリ・MCP を使いこなす

2026-06-14

- Claude Code 完全実践ガイド (#claude-code-完全実践ガイド)
 - スキル・フック・メモリ・MCP を使いこなす (#スキルフックメモリmcp-を使いこなす)
 - この教材の使い方（3ステップ） (#この教材の使い方3ステップ)
 - カテゴリー一覧 (#カテゴリー一覧)
 - 推奨学習ルート (#推奨学習ルート)
 - 本書で学べること (#本書で学べること)
 - 対象読者 (#対象読者)
- 0. 基礎と環境準備 (#基礎と環境準備)
 - 0.1 0-1: Claude Code とは (#claude-code-とは)
 - 0.2 0-2: インストールとセットアップ (#インストールとセットアップ)
- 1. コア機能 (#コア機能)
 - 1.1 1-1: チャットとファイル編集 (#チャットとファイル編集)
 - 1.2 1-2: スラッシュコマンド一覧 (#スラッシュコマンド一覧)
 - 1.3 1-3: ターミナル・Bash ツール (#ターミナルbash-ツール)
 - 1.4 1-4: コンテキスト管理 (#コンテキスト管理)
- 2. スキルシステム (#スキルシステム)
 - 2.1 2-1: スキルとは (#スキルとは)
 - 2.2 2-2: /skill-creator ハンズオン (#skill-creator-ハンズオン)
 - 2.3 2-3: 組み込みスキル活用 (#組み込みスキル活用)
 - 2.4 2-4: カスタムスキル作成 (#カスタムスキル作成)
- 3. フックと自動化 (#フックと自動化)
 - 3.1 3-1: フックの概要 (#フックの概要)
 - 3.2 3-2: フックの種類 (#フックの種類)

- 3.3 3-3: 自動化パターン (#自動化パターン)
- 3.4 3-4: settings.json 設定 (#settings.json-設定)
- 4. メモリシステム (#メモリシステム)
 - 4.1 4-1: メモリの種類 (#メモリの種類)
 - 4.2 4-2: CLAUDE.md ファイル (#claude.md-ファイル)
 - 4.3 4-3: 永続的なメモリ (#永続的なメモリ)
 - 4.4 4-4: メモリのベストプラクティス (#メモリのベストプラクティス)
- 5. MCP とエージェント (#mcp-とエージェント)
 - 5.1 5-1: MCP の概要 (#mcp-の概要)
 - 5.2 5-2: MCP サーバー設定 (#mcp-サーバー設定)
 - 5.3 5-3: エージェント SDK (#エージェント-sdk)
 - 5.4 5-4: マルチエージェント (#マルチエージェント)
 - 5.5 5-5: カスタムエージェント実装 (#カスタムエージェント実装)
- 6. 発展・応用 (#発展応用)
 - 6.1 6-1: CI/CD 統合 (#cicd-統合)
 - 6.2 6-2: チームワークフロー (#チームワークフロー)
 - 6.3 6-3: パフォーマンス Tips (#パフォーマンス-tips)
 - 6.4 6-4: トラブルシューティング (#トラブルシューティング)

Claude Code 完全実践ガイド

スキル・フック・メモリ・MCP を使いこなす

この教材は、Anthropic の AI コーディングアシスタント **Claude Code** を基礎から応用まで体系的に学ぶ実践ガイドです。スキルシステム・フック自動化・メモリ管理・MCP/エージェント連携の4大機能を完全カバーします。

💡 ブラウザで <https://duwenji.github.io/spa-quiz-app/> を開くと、関連トピックをクイズ形式で復習できます。

📖 全体ガイド | 前提: Claude Code インストール済み

🔄 **更新状況:** 🔄 Claude Code 最新バージョン対応中

最終更新: 2026年6月 | スキル・フック・MCP 1.0 対応

この教材の使い方（3ステップ）

- Part 0 で環境を整えて Claude Code を起動する
- Part 1～2 でコア機能とスキルシステムを体験する
- Part 3～6 で自動化・メモリ・エージェント連携まで習得する

カテゴリー一覧

#	Part	難易度	学習時間
0	基礎と環境準備	★	20分
1	コア機能	★ ★	60分
2	スキルシステム	★ ★	80分
3	フックと自動化	★ ★	60分
4	メモリシステム	★ ★	60分
5	MCP とエージェント	★ ★ ★	90分
6	発展・応用	★ ★ ★	65分

推奨学習ルート

- 最短で体験する: 0 → 1 → 2
- 自動化を極める: 3 → 4
- エージェント開発: 5 → 6

本書で学べること

- Claude Code の設計思想とアーキテクチャを理解できる
- スキルシステムでチームの作業を自動化できる
- フックで繰り返し作業をゼロにできる
- CLAUDE.md と永続メモリでコンテキストを管理できる
- MCP サーバーで外部ツールと連携できる

- カスタムエージェントを設計・実装できる

対象読者

- Claude Code を使い始めた開発者
 - スキルやフックで業務効率を上げたいエンジニア
 - AI コーディングアシスタントの全機能を体系的に習得したいチーム
-

0. 基礎と環境準備

0.1 0-1: Claude Code とは

学習時間: 10分 | 難易度: ★

0.1.1 概要

Claude Code は Anthropic が開発した AI コーディングアシスタントです。ターミナル上で動作する CLI ツールとして設計されており、コードの生成・編集・レビュー・デバッグをインタラクティブに行えます。

Claude Code は単なるチャットボットではなく、**実際にファイルを読み書きし、コマンドを実行できるエージェント**です。

0.1.2 設計思想

Claude Code の設計は3つの原則に基づいています。

1. エージェント的に動作する

ファイルの読み書き、ターミナルコマンドの実行、外部 API の呼び出しなど、実際に作業を完遂するために必要なアクションを自律的に取ります。

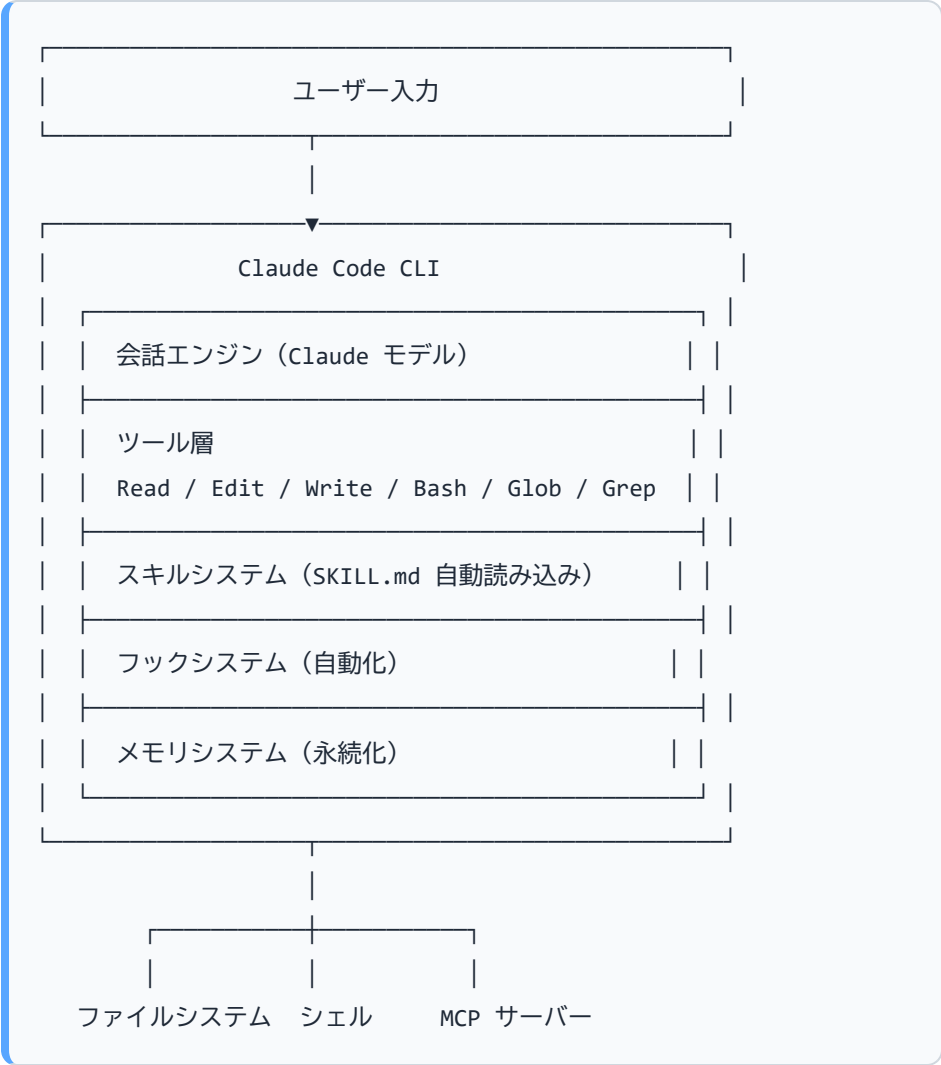
2. 人間の監督を前提とする

破壊的な操作（ファイル削除、force push など）は事前に確認を求めます。ユーザーが最終的な意思決定者です。

3. プロジェクトに馴染む

CLAUDE.md やメモリシステムを通じて、プロジェクト固有のルールや過去の文脈を記憶します。

0.1.3 アーキテクチャ



0.1.3.1 Windows でのコマンド実行

アーキテクチャ図の「Bash」はツール名であり、実際に使われるシェルは OS によって異なります。

環境	主要シェル	Bash ツール
macOS / Linux	bash / zsh	同じシェルを使用
Windows	PowerShell 7+	Git Bash（Git for Windows が必要）

Windows では **PowerShell が主要シェル**として動作するため、`git`・`npm`・`docker` などほぼすべての操作を PowerShell 経由で実行できます。Bash ツールは POSIX 系スクリプトを実行したい場合に補助的に使います（パス区切り文字はスラッシュ `/` が必要）。

0.1.4 他ツールとの比較

観点	Claude Code	GitHub Copilot	Cursor
動作場所	ターミナル / IDE 拡張	IDE 拡張	IDE（独自）
エージェント機能	フル（ファイル操作・コマンド実行）	限定的	限定的
スキルシステム	あり（SKILL.md）	なし	なし
フック・自動化	あり（hooks）	なし	なし
メモリ永続化	あり（ファイルベース）	なし	なし
MCP 対応	あり	なし	なし
オープンソース	CLI はオープン	クローズド	クローズド

0.1.5 利用できる環境

- **CLI:** ネイティブインストーラーまたは npm でインストール（詳細は次章）
- **VS Code 拡張:** Marketplace から「Claude Code」を検索
- **JetBrains プラグイン:** JetBrains Marketplace から入手
- **Desktop App:** Mac / Windows 対応
- **Web:** claude.ai/code

- **iOS アプリ**: Claude アプリ経由で利用可能

0.1.6 Claude Code が得意なこと

タスク	説明
コード生成	要件から実装コードを生成
リファクタリング	既存コードの改善・整理
デバッグ	エラーの原因特定と修正
コードレビュー	品質・セキュリティ・パフォーマンスの評価
テスト生成	ユニットテスト・統合テストの自動生成
ドキュメント作成	コメント・README の生成
Git 操作	コミット・PR 作成の補助

0.1.7 次のステップ

→ 0-2: インストールとセットアップ (02-installation-setup.md) に進む

0.2 0-2: インストールとセットアップ

学習時間: 10分 | 難易度: ★

0.2.1 前提条件

0.2.1.1 システム要件

項目	要件
OS	macOS 13.0+、Windows 10 1809+、Ubuntu 20.04+、Debian 10+、Alpine 3.19+
RAM	4GB 以上
ネットワーク	インターネット接続必須
Node.js	npm 経由でインストールする場合のみ 18 以上が必要

0.2.1.2 アカウント要件

Claude.ai の有料プラン（Pro / Max / Team / Enterprise）または Anthropic Console アカウントが必要です。**無料プランは Claude Code を利用できません。**

0.2.2 インストール

0.2.2.1 推奨: ネイティブインストーラー

ネイティブインストーラーは Node.js 不要で、バックグラウンドで自動更新されます。

macOS / Linux / WSL:

```
curl -fsSL https://claude.ai/install.sh | bash
```

Windows PowerShell:

```
irm https://claude.ai/install.ps1 | iex
```

Windows CMD:

```
curl -fsSL https://claude.ai/install.cmd -o install.cmd &&  
install.cmd && del install.cmd
```

0.2.2.2 その他のインストール方法

Homebrew (macOS) :

```
brew install --cask claude-code
```

WinGet (Windows) :

```
winget install Anthropic.ClaudeCode
```

npm (Node.js 18+ が必要) :

```
npm install -g @anthropic-ai/claude-code
```

Homebrew・WinGet・npm 経由のインストールは自動更新されません。手動で `brew upgrade claude-code` / `winget upgrade Anthropic.ClaudeCode` / `npm install -g @anthropic-ai/claude-code@latest` を実行して更新してください。

0.2.3 インストール確認

```
claude --version
```

```
# より詳細な診断
```

```
claude doctor
```

0.2.4 認証設定

0.2.4.1 方法1: Claude.ai アカウント（推奨・通常の使い方）

初回起動時にブラウザが開き、Claude アカウントでサインインします。

```
claude auth login
```

対応プランと特徴：

プラン	月額	Claude Code での利用
Max	\$100/\$200	レート制限が高く最適
Pro	\$20	利用可能（ヘビーな使用時に制限あり）
Team	\$25/人	チーム向け、管理機能あり
Enterprise	要相談	SSO・管理コンソール対応

その他の認証コマンド：

```
claude auth logout # ログアウト
```

```
claude auth status # 認証状態を確認
```

```
claude setup-token # CI 用の長期トークンを生成
```

0.2.4.2 方法2: Anthropic Console / API キー

Amazon Bedrock・Google Vertex AI など外部プロバイダー経由、または API キーで利用する場合：

```
export ANTHROPIC_API_KEY="sk-ant-..."
```

永続的に設定するには `~/.bashrc` / `~/.zshrc` に追加します。

注意: `ANTHROPIC_API_KEY` が設定されている場合、サブスクリプションより API キーが優先されます。

0.2.5 初回起動

プロジェクトのルートディレクトリで起動します：

```
cd my-project  
claude
```

初回起動時に認証方法の選択など、いくつかの設定確認が行われます。

0.2.6 IDE 拡張のインストール

0.2.6.1 VS Code

1. VS Code を開く
2. 拡張機能（Ctrl+Shift+X）を開く
3. 「Claude Code」を検索してインストール
4. コマンドパレット（Ctrl+Shift+P）から `Claude Code: Open` を実行

0.2.6.2 JetBrains

1. Settings → Plugins → Marketplace
2. 「Claude Code」を検索してインストール

0.2.7 CLAUDE.md の初期化

プロジェクトルートで `CLAUDE.md` を自動生成できます：

```
claude  
/init
```

Claude Code がプロジェクトを分析し、CLAUDE.md の初稿を作成します。

0.2.8 設定ファイルの場所

ファイル	場所	用途
グローバル設定	<code>~/.claude/settings.json</code>	全プロジェクト共通設定
プロジェクト設定	<code>.claude/settings.json</code>	プロジェクト固有設定
プロジェクト指示	<code>CLAUDE.md</code>	プロジェクトのコンテキスト
個人用スキル	<code>~/.claude/skills/</code>	全プロジェクトで使えるスキル
プロジェクト用スキル	<code>.claude/skills/</code>	そのプロジェクト専用スキル
メモリ	<code>~/.claude/projects/<hash>/memory/</code>	プロジェクト別の永続メモリ

0.2.9 モデルの選択

`settings.json` でモデルを指定できます：

```
{
  "model": "claude-opus-4-8"
}
```

利用可能なモデル：

モデル ID	特徴
claude-opus-4-8	最高性能、複雑なタスク向け
claude-sonnet-4-6	バランス型（デフォルト）
claude-haiku-4-5-20251001	高速・軽量

0.2.10 動作確認

```
# インストールと設定の診断
claude doctor

# 起動して動作確認
claude
> Hello! このプロジェクトの README.md を読んで概要を教えて
```

`claude doctor` はインストール状態・認証・ネットワーク接続などを診断し、問題があれば修正方法を提示します。Claude Code がファイルを読み込み、プロジェクトの概要を説明します。

0.2.11 次のステップ

→ 1-1: チャットとファイル編集 (../01-core-features/01-chat-and-editing.md) に進む

1. コア機能

1.1 1-1: チャットとファイル編集

学習時間: 15分 | 難易度: ★★

1.1.1 基本的な会話

Claude Code は対話形式で作業します。自然言語で指示を出すだけで、必要なツールを自律的に選択して実行します。

> このディレクトリにある TypeScript ファイルの一覧を教えてください

Claude: 確認します。

[Glob ツールを実行: **/*.ts]

```
src/  
├─ index.ts  
├─ components/Button.tsx  
└─ utils/helpers.ts
```

1.1.2 ファイル読み込み

Claude Code はプロジェクト内のファイルを直接読み込んで作業します。

> src/utils/helpers.ts を読んで、何をしているか説明して

明示的にファイルを指定しなくても、文脈から関連ファイルを自動的に読み込みます。

1.1.3 コード生成

- > React コンポーネント `Button.tsx` を作成して。
- > プロパティ: `label(string)`, `onClick(function)`, `disabled(boolean)`
- > スタイル: Tailwind CSS を使用

Claude Code は要件を解釈し、`Button.tsx` を作成します。

1.1.4 ファイル編集

- > `src/utils/helpers.ts` の `formatDate` 関数を
- > ISO 8601 形式に対応するよう修正して

Edit ツールを使って既存ファイルを差分で編集します（ファイル全体の書き直しはしません）。

1.1.5 リファクタリング

- > `UserList` コンポーネントのパフォーマンスを改善して。
- > 不要な再レンダリングを防ぐよう `useMemo` と `useCallback` を適切に使って

1.1.6 デバッグ補助

エラーメッセージをそのまま貼り付けるだけで原因を特定します：

```
> npm run build で以下のエラーが出ます：
> TypeError: Cannot read properties of undefined (reading 'map')
>   at UserList (src/components/UserList.tsx:23:18)
```

1.1.7 ファイル・フォルダの参照

@ 記法でファイルやフォルダを会話に含められます（IDE 拡張のみ）：

```
@src/components/Button.tsx をレビューして
@src/utils/ の全ファイルの依存関係を整理して
```

1.1.8 複数ファイルの同時編集

Claude Code は1回の指示で複数ファイルを同時に変更できます：

```
> Button コンポーネントの props に color プロパティを追加して。
> Button.tsx, Button.test.tsx, Storybook の Button.stories.tsx も
  更新して
```

1.1.9 ツール一覧

Claude Code が内部で使うツール：

ツール	用途
Read	ファイルの内容を読む
Edit	ファイルの一部を差分編集
Write	新しいファイルを作成
Glob	ファイルパターン検索
Grep	ファイル内容の全文検索
Bash	シェルコマンドを実行
Agent	サブエージェントを起動

1.1.10 承認フロー

破壊的な操作は実行前に確認を求めます：

Claude: 以下のファイルを削除しようとしています：

- src/legacy/old-api.ts

実行しますか？ [y/N]

`settings.json` の `permissions.allow` に追加すると自動承認できます。

1.1.11 次のステップ

→ 1-2: スラッシュコマンド一覧 (02-slash-commands.md) に進む

1.2 1-2: スラッシュコマンド一覧

学習時間: 15分 | 難易度: ★★

1.2.1 スラッシュコマンドとは

スラッシュコマンドは `/` で始まるショートカットで、特定の機能やスキルを即座に呼び出せます。組み込みコマンドと、スキルとして定義されたカスタムコマンドの2種類があります。

1.2.2 組み込みコマンド一覧

1.2.2.1 セッション管理

コマンド	説明
<code>/help</code>	利用可能なコマンドとヘルプを表示
<code>/clear</code>	会話履歴をクリアして新しい会話を開始
<code>/compact [instructions]</code>	会話を要約してコンテキストを節約
<code>/context</code>	コンテキスト使用状況を可視化
<code>/config</code>	設定を対話的に変更 (<code>/settings</code> でも可)
<code>/status</code>	バージョン・モデル・アカウント・接続状態を表示
<code>/fast on off</code>	Fast モード (Claude Opus 高速版) の切り替え
<code>/model [model]</code>	AI モデルを切り替え
<code>/effort [level]</code>	努力レベルを調整 (low / medium / high / xhigh / max)
<code>/exit</code>	Claude Code を終了
<code>/resume [session]</code>	過去のセッションを再開

コマンド	説明
<code>/rename [name]</code>	セッションに名前を付ける
<code>/branch [name]</code>	会話をブランチしてコピーを作成
<code>/rewind</code>	チェックポイントに巻き戻す
<code>/export [filename]</code>	会話をテキストファイルに書き出す

1.2.2.2 ファイル・プロジェクト

コマンド	説明
<code>/init</code>	CLAUDE.md を自動生成（プロジェクト分析）
<code>/memory</code>	CLAUDE.md とオートメモリを管理
<code>/diff</code>	インタラクティブな差分ビューを開く
<code>/add-dir <path></code>	セッションに作業ディレクトリを追加

1.2.2.3 並列処理・バックグラウンド

コマンド	説明
<code>/agents</code>	サブエージェントの設定を管理
<code>/tasks</code>	バックグラウンドで実行中のタスクを表示
<code>/background [prompt]</code>	セッションをバックグラウンドに移行してターミナルを解放
<code>/batch <instruction></code>	大規模変更を並列サブエージェントで実行
<code>/schedule [description]</code>	ルーティン（定期実行タスク）を設定

1.2.2.4 コードレビュー・品質

コマンド	説明
<code>/code-review [--fix] [--comment]</code>	コードレビュー（ <code>--fix</code> で自動修正）
<code>/simplify [target]</code>	コードの簡略化・リファクタリングを適用
<code>/security-review</code>	セキュリティ脆弱性を分析
<code>/review [PR]</code>	プルリクエストをローカルでレビュー
<code>/ultrareview [PR]</code>	クラウドでのディープレビュー（ <code>/code-review ultra</code> でも可）

1.2.2.5 スキル・ツール

コマンド	説明
<code>/run-skill-generator</code>	<code>/run</code> や <code>/verify</code> 用のスキルを生成
<code>/run</code>	アプリを起動して動作確認
<code>/verify</code>	コード変更が実際に動作することを確認
<code>/debug [description]</code>	デバッグログを有効化して問題を診断
<code>/skills</code>	利用可能なスキルの一覧を表示

1.2.2.6 設定・診断

コマンド	説明
<code>/doctor</code>	インストールと設定を診断
<code>/permissions</code>	ツール許可ルールを管理
<code>/hooks</code>	フック設定を確認
<code>/mcp</code>	MCP サーバーの接続を管理
<code>/plan [description]</code>	プランモードに入る
<code>/ide</code>	IDE 連携の状態を確認・管理
<code>/feedback [report]</code>	フィードバックやバグを報告

1.2.3 コマンドの使い方

1.2.3.1 引数なし

```
/code-review
```

カレントブランチの変更差分をレビューします。

1.2.3.2 引数あり

```
/code-review --fix  
/code-review src/components/Button.tsx
```

1.2.3.3 スキルとして呼び出す

独自スキルも同様にスラッシュで呼び出せます：

```
/my-custom-skill
```

1.2.4 /init の使い方

プロジェクト初回セットアップ時に使います：

```
claude  
/init
```

Claude Code がプロジェクトを分析して `CLAUDE.md` を生成します。生成される内容： - プロジェクト概要 - 技術スタック - よく使うコマンド - コーディング規約の推測

1.2.5 /config の使い方

```
/config
```

対話的に設定できる項目： - モデルの選択 - テーマ（light / dark） - 自動コンパクト設定 - 通知設定

1.2.6 カスタムスラッシュコマンド

`.claude/skills/<name>/SKILL.md` を配置すると `/name` で呼び出せます：

```
.claude/skills/  
└─ deploy-check/  
    └─ SKILL.md    → /deploy-check として使える
```

1.2.7 /help の出力例

実際のコマンド一覧はバージョンや環境によって異なります。`/` を入力すると補完リストが表示されます。

<code>/help</code>	Show help and available commands
<code>/clear</code>	Start a new conversation
<code>/compact</code>	Free up context by summarizing
<code>/config</code>	Open the Settings interface
<code>/init</code>	Initialize project with a CLAUDE.md guide
<code>/model</code>	Switch the AI model
<code>/effort</code>	Set the model effort level
<code>/code-review</code>	Review the current diff
<code>/run</code>	Launch and drive your project's app
<code>/verify</code>	Confirm a code change works
<code>/debug</code>	Enable debug logging
<code>/doctor</code>	Diagnose your Claude Code installation
<code>/memory</code>	Edit CLAUDE.md memory files
<code>/agents</code>	Manage agent configurations
...	

Custom skills:

`/deploy-check` (from `.claude/skills/deploy-check/`)

1.2.8 次のステップ

→ 1-3: ターミナル・Bash ツール (03-terminal-bash.md) に進む

1.3 1-3: ターミナル・Bash ツール

学習時間: 15分 | 難易度: ★★

1.3.1 Bash ツールとは

Claude Code は Bash ツールを通じてシェルコマンドを実行できます。これにより、ビルド・テスト・デプロイなどの実際の作業を自律的に行えます。

1.3.2 基本的な使い方

＞ npm のテストを実行して、失敗しているテストを教えて

Claude Code は内部で以下を実行します：

```
npm test
```

出力を解析して失敗しているテストを特定し、修正案を提示します。

1.3.3 Git 操作

＞ 変更内容を確認してコミットメッセージを提案して

```
# Claude Code が実行する一連のコマンド
git status
git diff
git diff --staged
# → コミットメッセージを提案
```

＞ 現在のブランチの変更を main にマージして、コンフリクトがあれば解決して

1.3.4 ビルドとテスト

＞ ビルドして、エラーがあれば修正して

```
npm run build
```

```
# エラーを解析 → 関連ファイルを修正 → 再ビルド
```

1.3.5 パッケージ管理

> date-fns をインストールして、既存の日付処理を date-fns に置き換えて

```
npm install date-fns
```

```
# → 関連ファイルを自動編集
```

1.3.6 権限とセキュリティ

1.3.6.1 自動承認のルール

デフォルトでは危険な操作は確認が必要です。 `settings.json` で細かく制御できます：


```
{
  "permissions": {
    "allow": [
      "Bash(git *)",
      "Bash(npm run *)",
      "Bash(npm test)",
      "Read(**)",
      "Edit(**)"
    ],
    "deny": [
      "Bash(rm -rf *)",
      "Bash(git push --force *)"
    ]
  }
}
```

1.3.6.2 パターン構文

パターン	意味
<code>Bash(git *)</code>	git で始まる全コマンドを許可
<code>Bash(npm run *)</code>	npm run から始まる全コマンドを許可
<code>Read(**)</code>	全ファイルの読み込みを許可
<code>Bash(rm -rf *)</code>	rm -rf を拒否

1.3.7 バックグラウンド実行

長時間かかるコマンドはバックグラウンドで実行できます：

> 開発サーバーをバックグラウンドで起動して、起動完了を確認して

```
npm run dev &
```

起動ログを監視 → "Server running" を検出 → 確認完了

1.3.8 複数コマンドの連携

> 依存関係をインストールして、ビルドして、テストを実行して、全部成功したら git commit して

```
npm install && npm run build && npm test && git add -A && git  
commit -m "..."
```

Claude Code は各ステップの結果を確認し、失敗した場合は対処してから次に進みます。

1.3.9 よく使うパターン

1.3.9.1 プロジェクトの現状把握

> このプロジェクトのテストカバレッジを計測して

```
> npm run test:coverage
```

1.3.9.2 リント・フォーマット

> ESLint と Prettier を実行して、自動修正できるものは直して

```
> npm run lint:fix && npm run format
```

1.3.9.3 ポート確認

> 3000番ポートを使っているプロセスを教えて

1.3.10 次のステップ

→ 1-4: コンテキスト管理 (04-context-management.md) に進む

1.4 1-4: コンテキスト管理

学習時間: 15分 | 難易度: ★★

1.4.1 コンテキストとは

Claude Code の会話ウィンドウには上限（コンテキストウィンドウ）があります。長い作業では会話が圧縮・要約されることがあります。効果的なコンテキスト管理が生産性を左右します。

1.4.2 コンテキストの確認

```
/status
```

現在のトークン使用量と残量が表示されます。

1.4.3 コンテキストのリセット

```
/clear
```

会話履歴をクリアして新しいセッションを開始します。CLAUDE.md やメモリファイルの内容は保持されます。

1.4.4 自動コンパクト

コンテキストが一定量に達すると、Claude Code は自動的に過去の会話を要約して圧縮します。重要な決定事項やコードの変更は保持されます。

1.4.5 CLAUDE.md による常時コンテキスト

`CLAUDE.md` はセッション開始時に常に読み込まれます。ここに書かれた内容は `/clear` 後も有効です：

```
# My Project

## 技術スタック
- React 18 + TypeScript
- Tailwind CSS
- Vitest

## よく使うコマンド
- `npm run dev` - 開発サーバー（ポート 3000）
- `npm test` - テスト実行

## コーディング規約
- コンポーネントは関数コンポーネントのみ
- Props 型は interface で定義
- テストは Vitest + Testing Library
```

1.4.6 ファイルの明示的な読み込み

大きなプロジェクトでは、関連ファイルだけを指示して読み込ませると効率的です：

```
> src/auth/ ディレクトリだけ読んで、認証フローを説明して
```

> package.json と tsconfig.json を読んで、プロジェクトの設定を確認して

1.4.7 サブエージェントによるコンテキスト保護

長い調査タスクは Agent（サブエージェント）に委ねることでメインのコンテキストを消費せずに済みます：

> src/components/ 以下の全コンポーネントを調査して、
> props の型定義に不整合がないか確認するエージェントを起動して

1.4.8 セッションをまたいだ継続

セッションが終わっても作業を継続するには：

1. **CLAUDE.md** に進行状況を書いておく
2. **メモリスistem** に重要な決定を保存する
3. **コメント** として残す（ただし不要なコメントは削除）

1.4.9 効果的なコンテキスト管理のコツ

方法	効果
CLAUDE.md に技術スタックを書く	毎回説明不要
<code>/clear</code> を適切に使う	コンテキスト汚染を防ぐ
大きなタスクを分割する	1セッションの集中度を高める
メモリに決定事項を保存する	セッション間の継続性を確保
不要なファイルを読ませない	トークンを節約

1.4.10 次のステップ

→ 2-1: スキルとは (../02-skills/01-what-are-skills.md) に進む

2. スキルシステム

2.1 2-1: スキルとは

学習時間: 15分 | 難易度: ★★

2.1.1 概要

スキル（Skills）は、Claude Code に特定のタスクを実行させるための指示書です。`SKILL.md` というファイルに手順・ルール・出力形式を記述し、Claude Code が自動的に読み込んで実行します。

スキルは **Agent Skills オープンスタンダード** (agentskills.io (<https://agentskills.io>)) に基づく共通フォーマットで、Claude Code と GitHub Copilot の両方で使えます。

2.1.2 スキルの仕組み

ユーザー: 「このコードをレビューして」

|

▼

Claude Code が description を検索

|

▼

.claude/skills/code-review/SKILL.md を発見

|

▼

SKILL.md の手順に従ってレビュー実行

|

▼

構造化された出力を返す

2.1.3 SKILL.md の構造

```
---
name: code-review
description: コードの品質・セキュリティ・パフォーマンスをレビューしま
す
tags:
  - review
  - quality
---

# コードレビュースキル

## 概要
指定されたコードを複数の観点でレビューし、改善点を提示します。

## 手順
1. コードを読み込む
2. 品質・セキュリティ・パフォーマンスの観点でチェック
3. 優先度付きで改善点を列挙

## 出力形式
...
```

2.1.4 フロントマターの役割

フィールド	役割
<code>name</code>	スキルの識別子。ディレクトリ名と一致させる
<code>description</code>	自動読み込みの判断に使う説明文。具体的に書く
<code>tags</code>	カテゴリ分類（検索・発見に使用）

`description` が特に重要です。Claude Code はこのフィールドを見て「今の作業にこのスキルが必要か」を判断します。

2.1.5 スキルの配置場所

種類	パス	適用範囲
個人用	<code>~/.claude/skills/<name>/SKILL.md</code>	全プロジェクト
プロジェクト用	<code>.claude/skills/<name>/SKILL.md</code>	そのプロジェクトのみ

2.1.6 スキルが起動するタイミング

1. 自動読み込み

会話の内容がスキルの `description` にマッチすると自動的に読み込まれます。

2. スラッシュコマンドで明示呼び出し

```
/code-review
/debug
/my-custom-skill
```

2.1.7 ディレクトリ構造

```
.claude/skills/
└─ code-review/
    │ SKILL.md           # メインの指示（必須）
    │ scripts/          # 補助スクリプト（任意）
    │   └─ check.sh
    │ references/        # 参照ドキュメント（任意）
    │   └─ style-guide.md
    └─ assets/           # テンプレート等（任意）
        └─ report.md
```

2.1.8 組み込みスキル vs カスタムスキル

種類	説明	例	適用場面
組み込みスキル	Claude Code に標準搭載	<code>/code-review</code> , <code>/debug</code> , <code>/run</code>	設定不要ですぐ使える
バンドルスキル	<code>~/.claude/skills/</code> に追加済み	<code>/skill-creator</code>	インストール後に全プロジェクトで使える
プロジェクトスキル	<code>.claude/skills/</code> に自分で追加	<code>/deploy-check</code>	チームで共有したいワークフロー
個人スキル	<code>~/.claude/skills/</code> に自分で追加	<code>/my-workflow</code>	自分だけのカスタム手順

2.1.8.1 組み込みスキル

Claude Code にあらかじめ搭載されているスキルです。インストール直後から `/code-review` や `/debug` などがすぐに使えます。設定・追加作業は一切不要です。

2.1.8.2 バンドルスキル

Claude Code のインストール時に `~/.claude/skills/` に同梱されるスキルです。`/skill-creator` のように、スキルそのものを作るためのスキルが含まれます。全プロジェクトで共通して利用できます。

2.1.8.3 プロジェクトスキル

リポジトリ内の `.claude/skills/` に配置するスキルです。デプロイ手順やチーム固有のレビュー基準など、**プロジェクト固有のワークフロー**を定義するのに適しています。git で管理することでチームメンバー全員が同じスキルを共有できます。

2.1.8.4 個人スキル

`~/.claude/skills/` に自分で追加するスキルです。よく使う個人的な作業手順やフォーマットなど、**自分だけのカスタムワークフロー**を定義します。すべてのプロジェクトに横断して適用されます。

2.1.9 GitHub Copilot との互換性

同じ SKILL.md を `.github/skills/<name>/SKILL.md` に置くと GitHub Copilot でも使えます。Agent Skills オープンスタンダードに準拠しているためです。

2.1.10 次のステップ

→ 2-2: `/skill-creator` ハンズオン (`02-skill-creator.md`) に進む

2.2 2-2: /skill-creator ハンズオン

学習時間: 20分 | 難易度: ★★

2.2.1 skill-creator とは

`/skill-creator` は Claude Code に標準搭載されているスキル生成スキルです。対話形式でスキルの要件を引き出し、SKILL.md を自動生成します。

生成されたスキルは自動的にテストされ、改善サイクルが回ります。

2.2.2 起動方法

```
claude
/skill-creator
```

または自然言語でも起動できます：

- ＞ コードレビュースキルを作って
- ＞ デプロイ前チェックのスキルを作成して

2.2.3 対話フロー

skill-creator を起動すると、以下の対話が始まります：

Claude: どんなスキルを作りますか？以下を教えてください：

1. このスキルに何をさせたいですか？
2. どのようなタイミングで起動すべきですか？
3. 出力形式の希望はありますか？
4. テストケースを2～3個教えてください。

2.2.3.1 対話例

あなた: PR のレビュー前にセキュリティチェックをするスキルを作って。
SQL インジェクション、XSS、認証漏れの3点を必ずチェックしたい。
出力は Markdown の表形式で、重大度ごとに分類して。

Claude: ありがとうございます。以下の要件で SKILL.md を作成します:

- 名前: security-check
- チェック項目: SQL インジェクション / XSS / 認証漏れ
- 出力: Markdown 表形式 (重大度別)
- 自動起動: "セキュリティ" "脆弱性" "PR レビュー" という言葉で

SKILL.md を生成中...

.claude/skills/security-check/SKILL.md を作成しました。

2.2.4 内部プロセス（8フェーズ）

フェーズ	内容
1. 意図のヒアリング	要件を対話で引き出す
2. SKILL.md 作成	ベストプラクティスに従って生成
3. テストケース作成	2～3個の現実的なテストプロンプトを生成
4. 並列実行	スキルあり / スキルなしで同時実行
5. 評価	定量的アサーション + 定性的レビュー
6. 反復改善	フィードバックに基づいてスキルを改善
7. Description 最適化	自動トリガーの精度を向上
8. パッケージング	<code>.skill</code> ファイルとして出力（任意）

2.2.5 生成後の確認と編集

```
cat .claude/skills/security-check/SKILL.md
```

生成された SKILL.md を確認し、必要に応じて手動で調整します。

2.2.6 テストの実行

```
/security-check
```

- > 以下の Python コードをセキュリティチェックして：
- > [コードを貼り付け]

期待通りに動作しているか確認します。

2.2.7 生成されたスキルの動作

`/security-check` を起動すると、スキルは以下の順で処理を行います：

- 1. **入力受け取り** — ユーザーが貼り付けたコードを対象として取得する
- 2. **チェック項目の走査** — SKILL.md に定義された順（SQL インジェクション → XSS → 認証漏れ）で静的解析を実行する
- 3. **重大度の分類** — 検出した問題を High / Medium / Low に分類する
- 4. **修正例の生成** — 検出した脆弱性に対して安全なコード例を提示する
- 5. **結果の出力** — Markdown 表形式でまとめる

実際の出力例（test-01 の Python コードを入力した場合）：

```
## セキュリティチェック結果

| 重大度 | 種別 | 箇所 | 内容 | 修正例 |
|-----|-----|-----|-----|-----|
| ● High | SQL インジェクション | `get_user()` L2 | 文字列連結でクエリを構築している | `db.execute("SELECT * FROM users WHERE name = ?", [username])` |
| ✔ 検出なし | XSS | - | - | - |
| ✔ 検出なし | 認証漏れ | - | - | - |
```

test-02 の JavaScript コード（`innerHTML` 使用）を入力した場合は XSS が検出されます：

セキュリティチェック結果

重大度	種別	箇所	内容	修正例
-----	-----	-----	-----	-----
🟢 検出なし	SQL インジェクション	-	-	-
🔴 High	XSS	`renderComment()` L2	innerHTML にユーザー入力を直接代入している	`element.textContent = comment` または DOMPurify を使用
🟢 検出なし	認証漏れ	-	-	-

このスキルは SKILL.md の指示に従って動作します。チェック項目・出力形式・重大度の定義はすべて SKILL.md に記述されており、編集することで動作を変更できます。

2.2.8 スキルの改善サイクル

- > security-check スキルを改善して。
- > CSRF チェックも追加して、出力に「修正の優先度」列も追加して

skill-creator を再度呼んで反復改善できます：

```
/skill-creator
> security-check スキルに CSRF チェックを追加して
```

2.2.9 生成されるファイルの例

```
.claude/skills/security-check/  
├─ SKILL.md          # メイン指示書（自動生成）  
└─ tests/  
    ├─ test-01-sql-injection.md  
    ├─ test-02-xss.md  
    └─ test-03-auth.md
```

2.2.10 テストケースの書き方（フォーマット例）

skill-creator が自動生成するテストケースは以下の形式です。手動で追加・編集することもできます。

```
tests/test-01-sql-injection.md
```

テストケース: SQL インジェクション検出

入力プロンプト

以下の Python コードをセキュリティチェックしてください:

```
\``python
def get_user(username):
    query = "SELECT * FROM users WHERE name = '" + username + "'"
    return db.execute(query)
\``
```

期待する出力（アサーション）

- [] SQL インジェクションの脆弱性を ****高**** 重大度で検出すること
- [] 修正例（パラメータ化クエリ）を提示すること
- [] Markdown 表形式で出力されること
- [] XSS・認証漏れは「検出なし」と明記すること

tests/test-02-xss.md

テストケース: XSS 検出 (クリーンコードとの比較)

入力プロンプト

以下の JavaScript をセキュリティチェックしてください:

```
\``javascript
function renderComment(comment) {
    document.getElementById('output').innerHTML = comment;
}
\``
```

期待する出力 (アサーション)

- [] XSS の脆弱性を ****高**** 重大度で検出すること
- [] `textContent` または DOMPurify を使った修正例を提示すること
- [] SQL インジェクション・認証漏れは「検出なし」と明記すること

2.2.11 評価フェーズの出力例

評価フェーズでは、skill-creator が各テストケースの「入力プロンプト」を実際にスキルへ渡して実行し、返ってきた出力がアサーションを満たしているかを自動判定します。

PASS / FAIL の意味 - PASS = スキルがアサーション通りに動作した (例: 脆弱なコードを正しく検出できた) - FAIL = スキルの出力がアサーションを満たさなかった (検出漏れ・形式違いなど)

テストケース実行後、skill-creator が自動でスキルあり／なしを比較評価します:

テストケース評価結果: security-check

【test-01-sql-injection】

- | | |
|------------------------|------|
| ✓ SQL インジェクションを高重大度で検出 | PASS |
| ✓ パラメータ化クエリの修正例あり | PASS |
| ✓ Markdown 表形式で出力 | PASS |
| ✓ XSS・認証漏れ「検出なし」の明記 | PASS |

スコア: 4/4

【test-02-xss】

- | | |
|----------------------------------|--------------|
| ✓ XSS を高重大度で検出 | PASS |
| ✗ textContent / DOMPurify の修正例なし | FAIL ← 改善が必要 |
| ✓ Markdown 表形式で出力 | PASS |

スコア: 2/3

総合スコア: 6/7 (86%)

改善提案: test-02 の修正例に DOMPurify の使用例を追加することを推奨します。

SKILL.md を自動修正しますか? [y/n]

y を入力すると SKILL.md が自動修正され、再テストが走ります。

2.2.12 Tips

- **具体的に伝える:** 「セキュリティチェック」より「SQL インジェクション・XSS・CSRF の3点をチェックして Markdown 表で出力」のほうが精度が高い
- **テストケースを先に考える:** どんな入力でどんな出力を期待するか、最初に考えておくと生成品質が上がる

- **description** は長めに: 自動トリガーの精度が上がる

2.2.13 次のステップ

→ 2-3: 組み込みスキル活用 (03-built-in-skills.md) に進む

2.3 2-3: 組み込みスキル活用

学習時間: 20分 | 難易度: ★★

2.3.1 組み込みスキル一覧

Claude Code には以下のスキルが標準搭載されています。

スキル	コマンド	用途
code-review	<code>/code-review</code>	コードレビュー
debug	<code>/debug</code>	デバッグ支援
run	<code>/run</code>	アプリ起動・動作確認
security-review	<code>/security-review</code>	セキュリティレビュー
simplify	<code>/simplify</code>	コード簡略化
skill-creator	<code>/skill-creator</code>	スキル生成
init	<code>/init</code>	CLAUDE.md 生成

2.3.2 /code-review

現在のブランチ差分またはファイルをレビューします。

```
# ブランチ全体をレビュー
/code-review

# 特定ファイルをレビュー
/code-review src/components/Button.tsx

# レビューして自動修正
/code-review --fix

# PR にインラインコメントを投稿
/code-review --comment
```

レビュー観点： - 正確性（バグ・ロジックエラー） - 再利用性・簡潔さ - パフォーマンス - セキュリティ

2.3.3 /debug

エラーや不具合の原因を特定して修正します。

```
# エラーメッセージを貼って実行
/debug

# または自然言語で
> /debug TypeError: Cannot read properties of undefined at
  userList.tsx:23
```

デバッグプロセス： 1. エラーメッセージ・スタックトレースを解析 2. 関連ファイルを読み込む 3. 根本原因を特定 4. 修正案を提示・適用

2.3.4 /run

アプリを起動して実際に動作を確認します。

```
/run
```

プロジェクトの種類（React, Node.js, Python 等）を自動検出し、適切な起動コマンドを実行します。フロントエンドの変更はブラウザで確認し、スクリーンショットを取得します。

2.3.5 /security-review

現在のブランチ差分のセキュリティ上の問題を検出します。

```
/security-review
```

チェック項目： - SQL インジェクション - XSS（クロスサイトスクリプティング） - CSRF - 認証・認可の問題 - 機密情報のハードコード - 依存パッケージの脆弱性

2.3.6 /simplify

変更されたコードの重複・複雑さ・非効率を検出し、自動改善します。

```
/simplify
```


改善観点： - 重複コードの統合 - 不要な抽象化の除去 - 読みやすさの向上 - パフォーマンス改善

2.3.7 /code-review ultra（マルチエージェントレビュー）

複数のサブエージェントが並列でレビューを行う高精度モードです：

```
/code-review ultra
```

```
# GitHub PR 番号を指定
```

```
/code-review ultra 42
```

通常の `/code-review` より深い分析が行われますが、時間とトークンを消費します。

2.3.8 スキルのカスタマイズ

組み込みスキルは上書きできます。`.claude/skills/code-review/SKILL.md`を作成すると、プロジェクト固有の動作に変更できます：

```
# プロジェクト専用 コードレビュー
```

```
## 追加チェック項目
```

- Tailwind クラスの順序（Prettier プラグインに準拠）
- コンポーネントの最大行数（100行以下）
- テストファイルの同時更新確認

```
（以下、標準のレビュー観点...）
```

2.3.9 次のステップ

→ 2-4: カスタムスキル作成 (04-custom-skills.md) に進む

2.4 2-4: カスタムスキル作成

学習時間: 25分 | 難易度: ★★

2.4.1 手書き SKILL.md の基本

`/skill-creator` を使わず、テキストエディタで直接 SKILL.md を作成することもできます。シンプルなスキルや、細かい制御が必要な場合に適しています。

2.4.2 SKILL.md テンプレート

```
---
name: deploy-check
description: デプロイ前に環境変数・ビルド・テストをチェックします。
deploy, production, リリースという言葉で起動します。
tags:
  - deploy
  - ci
  - quality
---

# デプロイ前チェックスキル

## 概要
本番デプロイ前に必要なチェックを自動実行します。

## チェック項目
1. 必須環境変数の確認
2. ビルドの成功確認
3. テストの全パス確認
4. セキュリティスキャン

## 手順

### Step 1: 環境変数チェック
以下の環境変数が設定されているか確認する：
- `DATABASE_URL`
- `API_KEY`
- `NODE_ENV=production`

### Step 2: ビルド実行
```bash
```

```
npm run build
```

```
...
```

エラーがあれば報告し、修正を提案する。

### Step 3: テスト実行

```
```bash
```

```
npm test
```

```
...
```

失敗したテストがあれば一覧を表示する。

Step 4: 結果サマリー

以下の形式で結果を出力する：

チェック項目	結果	詳細
-----	-----	-----
環境変数	✓/✗	...
ビルド	✓/✗	...
テスト	✓/✗	...

出力形式

Markdown の表形式でチェック結果を表示する。全項目 ✓ の場合のみ「デプロイ可」と表示する。

2.4.3 配置とテスト

```
mkdir -p .claude/skills/deploy-check/
```

```
# SKILL.md を上記の内容で作成
```

```
# テスト実行
```

```
claude
```

```
/deploy-check
```

2.4.4 ベストプラクティス

2.4.4.1 description を充実させる

自動トリガーの精度は `description` フィールドに直結します：

悪い例

description: デプロイチェックをします

良い例

description: >

本番デプロイ前の事前チェックを実行します。

環境変数・ビルド・テスト・セキュリティを確認します。

"deploy", "デプロイ", "production", "リリース", "本番" という言葉が出たときに自動的に提案します。

2.4.4.2 明確な出力形式を定義する

曖昧な指示より、具体的な出力形式を定義するほうが一貫した結果が得られます：

出力形式

必ず以下の JSON 形式で出力すること：

```
```json
{
 "status": "ok" | "warning" | "error",
 "checks": [
 {
 "name": "チェック名",
 "result": "pass" | "fail" | "warn",
 "message": "詳細メッセージ"
 }
],
 "summary": "全体のサマリー"
}
```
```

2.4.4.3 失敗ケースを明示する

エラーハンドリング

- ファイルが見つからない場合：エラーを報告してスキップ
- コマンドが失敗した場合：エラーログ全文を表示
- タイムアウト（30秒以上）の場合：中断してユーザーに通知

2.4.4.4 補助ファイルの活用

複雑なスキルは補助スクリプトを分離できます：

```
.claude/skills/deploy-check/  
├─ SKILL.md  
├─ scripts/  
│   ├── check-env.sh  
│   └─ run-security-scan.sh  
└─ references/  
    └─ deployment-checklist.md
```

SKILL.md 内で参照する：

```
## 環境変数チェック  
`scripts/check-env.sh` を実行して環境変数の状態を確認する。
```

2.4.5 スキルの共有

2.4.5.1 チームで共有する方法

1. `.claude/skills/` をリポジトリに含める（`.gitignore` に追加しない）
2. `README.md` にスキルの説明を記載する

2.4.5.2 個人スキルとして保存する

全プロジェクトで使いたいスキルは個人スキルとして保存：

```
mkdir -p ~/.claude/skills/my-workflow/  
# SKILL.md を作成
```

2.4.6 次のステップ

→ 3-1: フックの概要 (`../03-hooks-automation/01-hooks-overview.md`) に進む

3. フックと自動化

3.1 3-1: フックの概要

学習時間: 15分 | 難易度: ★★

3.1.1 フック（Hooks）とは

フックは、Claude Code の特定のイベントに対して自動的にシェルコマンドを実行する仕組みです。Claude Code が行動するたびに、決められた処理を自動で挟み込みます。

3.1.2 フックで実現できること

| 用途 | 例 |
|----------|-------------------------|
| 自動フォーマット | ファイル編集後に Prettier を実行 |
| 自動テスト | コード変更後に npm test を実行 |
| 通知 | Claude が完了したら Slack に通知 |
| ログ記録 | 全ツール呼び出しをファイルに記録 |
| セキュリティ | 危険なコマンドをブロック |
| 品質ゲート | lint が通らなければ編集を拒否 |

3.1.3 フックのイベント種類

| イベント | 発火タイミング |
|------------------|---------------------------|
| PreToolUse | ツール実行の直前 |
| PostToolUse | ツール実行の直後 |
| Stop | Claude Code がレスポンスを完了したとき |
| Notification | 通知が発生したとき |
| UserPromptSubmit | ユーザーがメッセージを送信したとき |

3.1.4 基本的な設定方法

フックは `settings.json` に記述します：

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit",
        "hooks": [
          {
            "type": "command",
            "command": "npm run format"
          }
        ]
      }
    ]
  }
}
```

この例では、`Edit` ツール（ファイル編集）の直後に `npm run format` が自動実行されます。

3.1.5 フックの実行結果

フックのコマンドは、exit code で成功/失敗を伝えます：

| exit code | 意味 |
|------------------|------------------|
| 0 | 成功。処理を続行 |
| 1（PreToolUse のみ） | ツール実行をブロック |
| その他 | エラーとして記録するが処理は続行 |

3.1.6 フックの出力

フックのコマンドが標準出力に JSON を出力すると、Claude Code はその内容をコンテキストとして読み込みます：

```
#!/bin/bash
# フックスクリプト
echo '{"message": "テスト失敗: UserList.test.tsx"}'
```

Claude Code はこのメッセージを受け取り、次のアクションに反映します。

3.1.7 設定ファイルの場所

| ファイル | 適用範囲 |
|--|-----------------|
| <code>~/.claude/settings.json</code> | 全プロジェクト共通 |
| <code>.claude/settings.json</code> | そのプロジェクトのみ |
| <code>.claude/settings.local.json</code> | ローカル専用（git 管理外） |

3.1.8 次のステップ

→ 3-2: フックの種類 (02-hook-types.md) に進む

3.2 3-2: フックの種類

学習時間: 15分 | 難易度: ★★

3.2.1 PreToolUse

ツールが実行される直前に発火します。ツールへの入力は **stdin** から **JSON** として渡されます。ブロックするには `permissionDecision: "deny"` を含む JSON を stdout に出力します（exit code 2 でも stderr メッセージと共にブロック可能）。

3.2.1.1 設定例: rm -rf をブロック

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "command": "bash -c 'CMD=$(cat | jq -r
.tool_input.command); if echo \"$CMD\" | grep -q \"rm -rf\"; then
echo \"{\\\"hookSpecificOutput\\\":
{\\\"hookEventName\\\":\\\"PreToolUse\\\",\\\"permissionDecision\\
\\\":\\\"deny\\\",\\\"permissionDecisionReason\\\":\\\"rm -rf は危
険なためブロックしました\\\"}}\"; fi'"
          }
        ]
      }
    ]
  }
}
```

スクリプトファイルに切り出すと読みやすくなります（`.claude/hooks/block-rm.sh`）：

```
#!/bin/bash
COMMAND=$(cat | jq -r '.tool_input.command // empty')
if echo "$COMMAND" | grep -q 'rm -rf'; then
    jq -n '{
        hookSpecificOutput: {
            hookEventName: "PreToolUse",
            permissionDecision: "deny",
            permissionDecisionReason: "rm -rf は危険なためブロックしまし
た"
        }
    }'
```

```
fi
```

3.2.1.2 設定例: git push 前に確認

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash(git push*)",
        "hooks": [
          {
            "type": "command",
            "command": "scripts/pre-push-check.sh"
          }
        ]
      }
    ]
  }
}
```

3.2.2 PostToolUse

ツールが実行された直後に発火します。自動フォーマット・テスト実行に使用します。

3.2.2.1 設定例: ファイル編集後に Prettier を実行

ツールの入力 は `stdin` から `JSON` で取得します。Edit/Write ツールのファイルパスは `.tool_input.file_path` に入っています。

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "bash -c 'FILE=$(cat | jq -r
.tool_input.file_path); npx prettier --write \"${FILE}\"
2>/dev/null || true'"
          }
        ]
      }
    ]
  }
}
```


3.2.3.1 設定例: 完了したら通知音を鳴らす

```
{
  "hooks": {
    "Stop": [
      {
        "hooks": [
          {
            "type": "command",
            "command": "afplay /System/Library/Sounds/Glass.aiff"
          }
        ]
      }
    ]
  }
}
```


3.2.3.2 設定例: 完了時に Slack 通知

```
{
  "hooks": {
    "Stop": [
      {
        "hooks": [
          {
            "type": "command",
            "command": "curl -X POST $SLACK_WEBHOOK -d '{\"text\": \"Claude Code がタスクを完了しました\"}'"
          }
        ]
      }
    ]
  }
}
```

3.2.4 Notification

Claude Code が通知を発生させたとき（承認待ち・エラー等）に発火します。

3.2.4.1 設定例: 承認待ちを通知

```
{
  "hooks": {
    "Notification": [
      {
        "hooks": [
          {
            "type": "command",
            "command": "osascript -e 'display notification\n\"Claude Code が入力待ちです\" with title \"Claude Code\\\"'"
          }
        ]
      }
    ]
  }
}
```

3.2.5 UserPromptSubmit

ユーザーがメッセージを送信した直後に発火します。入力の前処理や検証に使います。

3.2.5.1 設定例: 入力をログに記録

UserPromptSubmit の stdin には `{"prompt": "...", "permission_mode": "..."}` の形式で入力が渡されます。

```
{
  "hooks": {
    "UserPromptSubmit": [
      {
        "hooks": [
          {
            "type": "command",
            "command": "bash -c 'PROMPT=$(cat | jq -r .prompt);
echo \"\$(date): $PROMPT\" >> ~/.claude/prompt-history.log'"
          }
        ]
      }
    ]
  }
}
```

3.2.6 matcher の書き方

| パターン | マッチするツール |
|-------------------|---------------------------------|
| "Edit" | Edit ツールのみ |
| "Edit\ Write" | Edit または Write (\ でエスケープして記述) |
| "Bash" | 全 Bash コマンド |
| "Bash(git *)" | git で始まる Bash のみ |
| "mcp__memory__.*" | memory サーバーの全ツール (正規表現) |
| " " または省略 | 全ツール |

英数字・`_`・`|` のみで構成されるパターンはパイプ区切りリストとして扱われます。それ以外の文字が含まれる場合は JavaScript 正規表現として解釈されます。

3.2.7 ツール入力・出力の取得

フックコマンドはツールの入力・出力を **stdin** から **JSON** として受け取ります（環境変数ではありません）。`jq` を使って値を抽出します：

```
# PreToolUse: ツールへの入力コマンドを取得
COMMAND=$(cat | jq -r '.tool_input.command // empty')

# PostToolUse: 編集されたファイルパスを取得
FILE=$(cat | jq -r '.tool_input.file_path // empty')

# UserPromptSubmit: ユーザーの入力テキストを取得
PROMPT=$(cat | jq -r '.prompt // empty')
```

3.2.8 環境変数

フックコマンド内で利用できるシステム環境変数：

| 変数 | 利用可能なイベント | 内容 |
|---------------------------------|--------------|--|
| <code>CLAUDE_ENV_FILE</code> | SessionStart | 環境変数を永続化するためのファイルパス |
| <code>CLAUDE_EFFORT</code> | ツール使用イベント | 努力レベル
(low / medium / high / xhigh / max) |
| <code>CLAUDE_CODE_REMOTE</code> | 全イベント | Web 環境で実行中の場合
<code>"true"</code> |

コマンド文字列内で使えるパスプレースホルダー：

| プレースホルダー | 内容 |
|-------------------------------------|--------------------|
| <code>\${CLAUDE_PROJECT_DIR}</code> | プロジェクトルートの絶対パス |
| <code>\${CLAUDE_PLUGIN_ROOT}</code> | プラグインのインストールディレクトリ |
| <code>\${CLAUDE_PLUGIN_DATA}</code> | プラグインの永続データディレクトリ |

3.2.9 次のステップ

→ 3-3: 自動化パターン (03-automation-patterns.md) に進む

3.3 3-3: 自動化パターン

学習時間: 15分 | 難易度: ★★

3.3.1 パターン1: コード品質の自動維持

ファイルを編集するたびに lint と format を自動実行するパターンです。

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "bash -c 'npx eslint --fix
\\$CLAUDE_TOOL_OUTPUT_PATH\" 2>/dev/null; npx prettier --write
\\$CLAUDE_TOOL_OUTPUT_PATH\" 2>/dev/null; true'"
          }
        ]
      }
    ]
  }
}
```

効果: Claude Code がファイルを編集するたびに自動でコードが整形されます。

3.3.2 パターン2: テスト自動実行

コードを変更するたびに関連テストを実行するパターンです。

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "scripts/run-related-tests.sh"
          }
        ]
      }
    ]
  }
}
```

```
#!/bin/bash
# scripts/run-related-tests.sh
CHANGED_FILE="$CLAUDE_TOOL_OUTPUT_PATH"
TEST_FILE="${CHANGED_FILE/src\\//src\\/}".test.ts

if [ -f "$TEST_FILE" ]; then
  npx vitest run "$TEST_FILE" 2>&1
  exit $?
fi
```

3.3.3 パターン3: Git 操作のガード

force push や main への直接 push をブロックするパターンです。

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "command": "scripts/git-guard.sh"
          }
        ]
      }
    ]
  }
}
```



```
#!/bin/bash
# scripts/git-guard.sh
INPUT="$CLAUDE_TOOL_INPUT"

# force push をブロック
if echo "$INPUT" | grep -q "push.*--force\|push.*-f"; then
    echo '{"error": "force push は禁止されています。レビューを教えてください。"}'
    exit 1
fi

# main への直接 push をブロック
if echo "$INPUT" | grep -qE "push.*origin main|push.*origin master"; then
    echo '{"error": "main への直接 push は禁止されています。PR を作成してください。"}'
    exit 1
fi
```

3.3.4 パターン4: 完了通知

長時間のタスクが完了したときに通知するパターンです。

3.3.4.1 macOS の場合

```
{
  "hooks": {
    "Stop": [
      {
        "hooks": [
          {
            "type": "command",
            "command": "osascript -e 'display notification \"タスクが完了しました\" with title \"Claude Code\" sound name \"Glass\"'"
          }
        ]
      }
    ]
  }
}
```

3.3.4.2 Windows の場合

```
{
  "hooks": {
    "Stop": [
      {
        "hooks": [
          {
            "type": "command",
            "command": "powershell -command \"[System.Console]::Beep(1000, 300)\""
          }
        ]
      }
    ]
  }
}
```

3.3.5 パターン5: 作業ログの記録

すべての Bash コマンドと結果をログに記録するパターンです。

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "command": "bash -c 'echo \"[$(date +%Y-%m-%d\\%H:%M:%S)] CMD: $CLAUDE_TOOL_INPUT\" >> ~/.claude/activity.log'"
          }
        ]
      }
    ]
  }
}
```

3.3.6 パターン6: 型チェックの自動実行

TypeScript ファイルを編集するたびに型チェックを実行するパターンです。

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "bash -c 'if echo\n\"$CLAUDE_TOOL_OUTPUT_PATH\" | grep -q \"\\.tsx\\\"?\"; then npx\n\tsc --noEmit 2>&1 | tail -20; fi'"
          }
        ]
      }
    ]
  }
}
```

3.3.7 複数フックの組み合わせ

同じイベントに複数のフックを登録できます：

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          { "type": "command", "command": "npx prettier --write\n\"$CLAUDE_TOOL_OUTPUT_PATH\" 2>/dev/null || true" },
          { "type": "command", "command": "npx eslint --fix\n\"$CLAUDE_TOOL_OUTPUT_PATH\" 2>/dev/null || true" }
        ]
      }
    ],
    "Stop": [
      {
        "hooks": [
          { "type": "command", "command": "afplay\n/System/Library/Sounds/Glass.aiff" }
        ]
      }
    ]
  }
}
```

3.3.8 次のステップ

→ 3-4: settings.json 設定 (04-settings-config.md) に進む

3.4 3-4: settings.json 設定

学習時間: 15分 | 難易度: ★★

3.4.1 settings.json の場所

| ファイル | 適用範囲 | 用途 |
|--|------------|---------------|
| <code>~/.claude/settings.json</code> | グローバル | 全プロジェクト共通 |
| <code>.claude/settings.json</code> | プロジェクト | チームで共有する設定 |
| <code>.claude/settings.local.json</code> | プロジェクトローカル | 個人設定（git 管理外） |

3.4.2 完全な設定例

```
{
  "model": "claude-sonnet-4-6",
  "theme": "dark",
  "permissions": {
    "allow": [
      "Bash(git *)",
      "Bash(npm run *)",
      "Bash(npm test)",
      "Bash(npx prettier *)",
      "Bash(npx eslint *)",
      "Read(**)",
      "Edit(**)",
      "Write(**)"
    ],
    "deny": [
      "Bash(rm -rf *)",
      "Bash(git push --force *)",
      "Bash(DROP TABLE *)"
    ]
  },
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "npx prettier --write\n\"$CLAUDE_TOOL_OUTPUT_PATH\" 2>/dev/null || true"
          }
        ]
      }
    ]
  }
}
```



```
    }
  ],
  "Stop": [
    {
      "hooks": [
        {
          "type": "command",
          "command": "echo 'Task completed'"
        }
      ]
    }
  ]
},
"env": {
  "NODE_ENV": "development",
  "LOG_LEVEL": "debug"
},
"mcpServers": {
  "filesystem": {
    "command": "npx",
    "args": ["-y", "@modelcontextprotocol/server-filesystem",
"/Users/me/projects"]
  }
}
}
```

3.4.3 各設定項目の説明

3.4.3.1 model

使用するモデルを指定します：

```
{
  "model": "claude-opus-4-8"
}
```

| モデル | 特徴 |
|---------------------------|--------------|
| claude-opus-4-8 | 最高性能 |
| claude-sonnet-4-6 | バランス型（デフォルト） |
| claude-haiku-4-5-20251001 | 高速・低コスト |

3.4.3.2 permissions

ツールの自動承認・拒否を設定します：

```
{
  "permissions": {
    "allow": ["Bash(git *)", "Read(**)"],
    "deny": ["Bash(rm -rf *)"]
  }
}
```

パターン書式:

| パターン例 | 意味 |
|------------------------------------|----------------------------------|
| <code>"Bash(git *)"</code> | <code>git</code> で始まる全 Bash コマンド |
| <code>"Bash(npm run build)"</code> | 特定のコマンドのみ |
| <code>"Read(**)"</code> | 全ファイルの読み取り |
| <code>"Edit(src/**)"</code> | src/ 以下のファイル編集のみ |

3.4.3.3 env

セッション中に使用する環境変数を設定します：

```
{
  "env": {
    "DATABASE_URL": "postgresql://localhost/mydb",
    "API_BASE_URL": "http://localhost:3001"
  }
}
```

3.4.3.4 theme

UI テーマを設定します：

```
{
  "theme": "dark"
}
```

選択肢: `"dark"` / `"light"` / `"auto"`

3.4.4 プロジェクト用 settings.json の例

チームで共有する最小設定：

```
{
  "permissions": {
    "allow": [
      "Bash(npm *)",
      "Bash(git status)",
      "Bash(git diff *)",
      "Bash(git log *)",
      "Bash(git add *)",
      "Bash(git commit *)"
    ]
  },
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          { "type": "command", "command": "npm run format
2>/dev/null || true" }
        ]
      }
    ]
  }
}
```

3.4.5 settings.local.json の活用

個人の API キーやパスなど、チームと共有しない設定は `settings.local.json` に書きます：

```
{
  "env": {
    "ANTHROPIC_API_KEY": "sk-ant-...",
    "PERSONAL_ACCESS_TOKEN": "ghp_..."
  },
  "hooks": {
    "Stop": [
      {
        "hooks": [
          { "type": "command", "command": "afplay
/System/Library/Sounds/Glass.aiff" }
        ]
      }
    ]
  }
}
```

3.4.6 /config コマンドで設定する

`settings.json` を手書きせず、対話的に設定することもできます：

```
/config
```

3.4.7 次のステップ

→ 4-1: メモリの種類 (../04-memory-system/01-memory-types.md) に進む

4. メモリシステム

4.1 4-1: メモリの種類

学習時間: 15分 | 難易度: ★★

4.1.1 メモリシステムとは

Claude Code のメモリシステムは、セッションをまたいで情報を保持するための仕組みです。会話が終わっても、重要な情報を次のセッションで利用できます。

4.1.2 4種類のメモリ

4.1.2.1 user（ユーザー情報）

ユーザーの役割・目標・技術レベル・好みを記録します。

```
---
name: user-role
description: ユーザーの役割と技術スタック
metadata:
  type: user
---
```

フルスタック開発者。フロントエンドは React/TypeScript が得意、バックエンドは Go を使用。Docker は使いこなしているが、Kubernetes は学習中。

用途: 説明の詳しさ、使う言語、前提とする知識レベルの調整。

4.1.2.2 feedback（フィードバック）

Claude への指示・修正・確認済みのアプローチを記録します。

```
---  
name: feedback-no-trailing-summary  
description: レスpons末尾のサマリー不要  
metadata:  
  type: feedback  
---
```

レスpons末尾に「～を行いました」というサマリーを書かない。
ユーザーは diff を直接確認できるので不要。

****Why:**** ユーザーが「末尾のサマリーは不要」と明示的に指示。

****How to apply:**** 全レスponsで適用。コード変更後のまとめも省略。

用途: 同じ修正を繰り返し求める手間を省く。

4.1.2.3 project（プロジェクト情報）

進行中の作業・目標・決定事項・期限を記録します。

```
---
name: project-auth-rewrite
description: 認証モジュール書き換えプロジェクト
metadata:
  type: project
---
```

認証モジュールを JWT から Session-based に移行中。

期限: 2026-06-30。

****Why:**** セキュリティ監査でセッション管理の不備が指摘された。

****How to apply:**** 認証関連の変更提案はセッション方式を前提にする。

用途: 長期プロジェクトの文脈を毎回説明せずに済む。

4.1.2.4 reference（参照情報）

外部システムのリンク・リポジトリ・ドキュメントの場所を記録します。

```
---
name: reference-bug-tracker
description: バグ追跡システムの場所
metadata:
  type: reference
---
```

バグは Linear の "Frontend" プロジェクトで管理。

デザインは Figma の "Product v3" ファイル。

用途: 外部ツールを毎回説明せずに参照できる。

4.1.3 メモリの保存場所

```
~/ .claude/projects/<project-hash>/memory/  
├─ MEMORY.md          # インデックス（全メモリのリスト）  
├─ user-role.md  
├─ feedback-no-summary.md  
├─ project-auth.md  
└─ reference-tools.md
```

`MEMORY.md` はインデックスとして機能し、各セッション開始時に読み込まれます。

4.1.4 メモリの使い方

4.1.4.1 保存する

＞ この設定を覚えておいて：テストは必ず Vitest を使うこと

Claude Code は自動的にメモリファイルを作成します。

4.1.4.2 確認する

＞ 今まで何を覚えているか教えて

4.1.4.3 削除する

＞ auth-rewrite プロジェクトのメモリを削除して

4.1.5 次のステップ

→ 4-2: CLAUDE.md ファイル (02-claude-md.md) に進む

4.2 4-2: CLAUDE.md ファイル

学習時間: 15分 | 難易度: ★★

4.2.1 CLAUDE.md とは

`CLAUDE.md` はプロジェクトルートに置く指示書です。Claude Code はセッション開始時（および `/clear` 後）に自動的に読み込みます。

CLAUDE.md に書いたことは、毎回会話で説明しなくても常に有効です。

4.2.2 自動生成

```
claude
/init
```

Claude Code がプロジェクトを分析して CLAUDE.md の初稿を自動生成します。生成後、手動で調整します。

4.2.3 推奨構造

プロジェクト名

概要

このプロジェクトの1～2行の説明。

技術スタック

- 言語: TypeScript 5.x
- フレームワーク: React 18
- テスト: Vitest + Testing Library
- スタイル: Tailwind CSS
- ビルド: Vite

よく使うコマンド

- `npm run dev` - 開発サーバー起動 (ポート 3000)
- `npm test` - テスト実行 (watch モード)
- `npm run build` - プロダクションビルド
- `npm run lint` - ESLint チェック
- `npm run format` - Prettier フォーマット

ディレクトリ構造

src/

| | |
|---------------|------------------|
| ├ components/ | # UI コンポーネント |
| ├ hooks/ | # カスタムフック |
| ├ utils/ | # ユーティリティ関数 |
| ├ types/ | # TypeScript 型定義 |
| └ api/ | # API クライアント |

コーディング規約

- コンポーネントは関数コンポーネントのみ (クラスコンポーネント禁止)
- Props の型は interface で定義 (type alias 禁止)
- テストファイルはコンポーネントと同じディレクトリに配置

- インポートは絶対パス（`@/` エイリアス）を使用

注意事項

- `src/legacy/` は触らない（リプレイス予定）
- 環境変数は `.env.local` に書く（`.env` は git 管理）
- API モックは `msw` を使用（`fetch` のモックは禁止）

4.2.4 CLAUDE.md を書く場所

複数の CLAUDE.md を階層的に配置できます：

```
my-project/
├─ CLAUDE.md          # プロジェクト全体の指示
├─ src/
│   └─ CLAUDE.md      # src/ 以下での作業時に追加読み込み
└─ docs/
    └─ CLAUDE.md      # docs/ 以下での作業時に追加読み込み
```

4.2.5 書くべき内容・書かない内容

| 書くべき | 書かない |
|--------------|--------------------------------------|
| 技術スタックとバージョン | コードの実装詳細（コードを読めばわかる） |
| よく使うコマンド | git 履歴（ <code>git log</code> で確認できる） |
| コーディング規約 | 一時的なタスクの状態 |
| 触ってはいけないファイル | 汎用的な開発常識 |
| 外部ツールの接続情報 | 既に CLAUDE.md に書いてある内容の繰り返し |

4.2.6 効果的な書き方のコツ

4.2.6.1 理由を添える

悪い例

- テストには Vitest を使う

良い例

- テストには Vitest を使う (Jest から移行済み。新しく Jest を追加しないこと)

4.2.6.2 禁止事項は明確に

禁止事項

- `any` 型の使用 (TypeScript の型安全性を損なう)
- `console.log` をコミットすること (デバッグ用のみ)
- `src/legacy/` の直接編集 (移行チームの担当)

4.2.6.3 バージョン依存を明示する

バージョン

- React 18.3.x (新しい Hooks は 18.3+ 対応のものを使う)
- Node.js 20.x (ESM モジュールを使用)

4.2.7 次のステップ

→ 4-3: 永続的なメモリ (03-persistent-memory.md) に進む

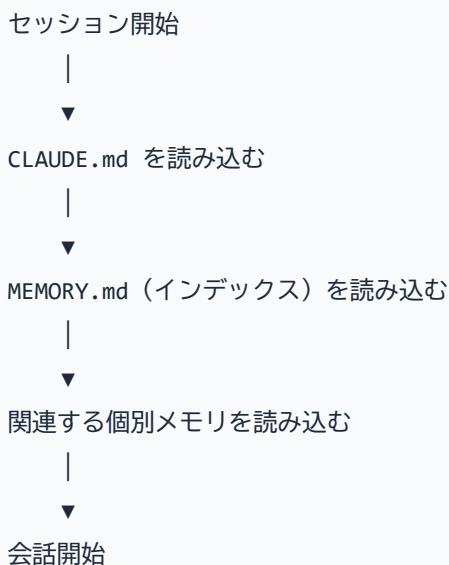
4.3 4-3: 永続的なメモリ

学習時間: 15分 | 難易度: ★★

4.3.1 セッションをまたいだ記憶

Claude Code のメモリシステムは `~/.claude/projects/<hash>/memory/` にファイルとして保存されます。新しいセッションを開始したときや `/clear` 後でも、このメモリは引き継がれます。

4.3.2 メモリの読み込みタイミング



4.3.3 メモリの保存・更新

4.3.3.1 自動保存

Claude Code が重要な情報を検出すると自動的にメモリを作成します：

> テストフレームワークは Vitest に統一することになりました

Claude: 了解しました。メモリに保存しておきます。

[memory/feedback-testing.md を作成]

4.3.3.2 手動で保存を依頼

> 「PR は必ず 1 機能ずつ分けてレビューしてもらう」という方針を覚えておいて

Claude: 保存しました。

[memory/project-pr-policy.md を作成]

4.3.4 MEMORY.md インデックス

各メモリへのポインタとなるインデックスファイルです。セッション開始時に必ず読み込まれるため、200行以内に収めるのが目安です：

Memory Index

ユーザー

- [user-role](user-role.md) – フルスタック開発者、React/TypeScript + Go 専門、Docker、Kubernetes 学習中

フィードバック

- [feedback-no-trailing-summary](feedback-no-trailing-summary.md)
- レスポンス末尾のサマリー不要 (diff を直接確認するため)
- [feedback-testing](feedback-testing.md) – テストフレームワークは Vitest のみ使用、Jest 禁止

プロジェクト

- [project-auth](project-auth.md) – 認証モジュール JWT→Session 移行中 (期限: 2026-06-30)
- [project-freeze](project-freeze.md) – 2026-06-20 以降マージ凍結 (モバイルリリース対応)

参照

- [reference-design](reference-design.md) – Figma "Product v3" ファイル、Linear "Frontend" プロジェクト

4.3.4.1 対応するメモリファイルのサンプル

MEMORY.md の各行が指すファイルはこのような内容になります：

feedback-testing.md

name: feedback-testing

description: テストフレームワークは Vitest のみ (Jest 使用禁止)

metadata:

type: feedback

テストは必ず Vitest を使う。Jest は使わない。

****Why:**** 昨年 Jest のモックが本番と挙動が異なり、リリース後に障害が発生した。

****How to apply:**** テストコードを書くとき・修正するとき全般に適用。

project-auth.md

name: project-auth

description: 認証モジュール書き換えプロジェクト (期限 2026-06-30)

metadata:

type: project

認証モジュールを JWT から Session-based に移行中。期限: 2026-06-30。

****Why:**** セキュリティ監査でセッション管理の不備が指摘された (コンプライアンス要件)。

****How to apply:**** 認証関連の変更はセッション方式を前提に提案する。エルゴノミクスより準拠を優先。

4.3.5 メモリの確認

> 今どんなことを覚えているか教えて

> feedback に関するメモリを全部見せて

4.3.6 メモリの更新

> auth プロジェクトの期限が 7 月末に変更になったので更新して

4.3.7 メモリの削除

> project-auth のメモリは完了したので削除して

4.3.8 メモリとして保存すべき情報

| 保存する | 保存しない |
|--------------|-------------------|
| ユーザーの技術的背景 | 実装の詳細（コードを見ればわかる） |
| 繰り返し出てきた要望 | 一時的なデバッグの結果 |
| プロジェクトの方針・制約 | 汎用的な技術情報 |
| 外部ツールのリンク | 現在の会話コンテキスト |
| 確認済みのアプローチ | git 履歴で追える情報 |

4.3.9 メモリファイルの手動編集

メモリファイルはただの Markdown ファイルなので、直接編集できます：

```
code ~/.claude/projects/<hash>/memory/feedback-testing.md
```

4.3.10 メモリの有効期限に注意

メモリは「書いたときに正しかった情報」です。コードやプロジェクトが変わっても、メモリは自動更新されません。定期的に見直して古くなったメモリを削除・更新しましょう。

＞ メモリの内容を確認して、現在の状況と合っているか確認して

4.3.11 次のステップ

→ 4-4: メモリのベストプラクティス (04-memory-best-practices.md) に進む

4.4 4-4: メモリのベストプラクティス

学習時間: 15分 | 難易度: ★★

4.4.1 メモリ設計の原則

4.4.1.1 原則1: 「なぜ」を必ず記録する

ルールだけを書いても、例外ケースで正しく適用できません：

悪い例

テストは必ず Vitest を使う。

良い例

テストは必ず Vitest を使う。

****Why:**** Jest から Vitest に移行済み。Jest の設定ファイルは残っているが使わない。

****How to apply:**** 新しいテストファイルを作るとき、package.json を変更するとき。

4.4.1.2 原則2: 適切な粒度で分割する

1つのメモリアイテムに詰め込みすぎない：

悪い例：1つのファイルに全部

memory/everything.md

良い例：トピックごとに分割

memory/feedback-testing.md

memory/feedback-pr-policy.md

memory/project-auth-rewrite.md

memory/reference-design-tools.md

4.4.1.3 原則3: 定期的に棚卸しする

月に1回、メモリの内容が現状に合っているか確認します：

＞ メモリの一覧を見せて。古くなっているものがあれば教えて

4.4.1.4 原則4: コードで追える情報は書かない

以下は CLAUDE.md やメモリに書く必要がありません： - どのファイルに何が書いてあるか（`grep` でわかる） - 最近の変更内容（`git log` でわかる） - コードの実装パターン（コードを読めばわかる）

4.4.2 フィードバックメモリの書き方

フィードバックは修正（×したこと）だけでなく、承認（✓したこと）も記録します：

```
---
name: feedback-single-pr
description: リファクタリングは1つのPRにまとめる方針
metadata:
  type: feedback
---
```

大規模リファクタリングは分割せず1つの PR にまとめる。

****Why:**** 小さく分割すると diff が追いにくくなることがユーザーが確認した。

****How to apply:**** リファクタリング系のタスクで PR 分割を提案しない。

4.4.3 CLAUDE.md とメモリの使い分け

| 情報の種類 | 置き場所 |
|-----------------|------------------|
| プロジェクト全体の技術スタック | CLAUDE.md |
| よく使うコマンド | CLAUDE.md |
| コーディング規約 | CLAUDE.md |
| ユーザーの技術的背景 | メモリ（user 型） |
| Claude への修正指示 | メモリ（feedback 型） |
| 進行中プロジェクトの状況 | メモリ（project 型） |
| 外部ツールのリンク | メモリ（reference 型） |

4.4.4 メモリが肥大化した場合

MEMORY.md の上限は200行です。不要なメモリを整理するか、複数のメモリをマージします：

＞ メモリが増えすぎているので、feedback 関連のものをまとめて整理して

4.4.5 チームでのメモリ活用

メモリはユーザーごとに独立しているため、チームメンバーと共有はできません。チーム共通の知識は CLAUDE.md に書きましょう。

個人の作業スタイルや好みをメモリに、チームの規約を CLAUDE.md に分離することが推奨されます。

4.4.6 次のステップ

→ 5-1: MCP の概要 ([../05-mcp-agents/01-mcp-overview.md](#)) に進む

5. MCP とエージェント

5.1 5-1: MCP の概要

学習時間: 15分 | 難易度: ★★ ★

5.1.1 MCP (Model Context Protocol) とは

MCP は Anthropic が策定したオープン標準プロトコルです。AI モデルと外部ツール・データソースを安全に接続するための共通インターフェースを定義しています。

Claude Code は MCP クライアントとして動作し、MCP サーバーを通じて外部システムと連携できます。

5.1.2 MCP でできること

| カテゴリ | 例 |
|-----------|---------------------------|
| ファイルシステム | 指定ディレクトリの読み書き |
| データベース | SQLite / PostgreSQL クエリ |
| バージョン管理 | GitHub Issue / PR 操作 |
| Web | URL フェッチ、検索 |
| コミュニケーション | Slack メッセージ送信 |
| クラウド | AWS / GCP / Azure 操作 |
| 開発ツール | Jira / Linear / Notion 操作 |

5.1.3 MCP のアーキテクチャ

Claude Code (MCP クライアント)

|

| MCP プロトコル

| (stdio / SSE)

|

▼

MCP サーバー

|

▼

外部システム

(GitHub / DB / Slack 等)

5.1.4 MCP が提供する3種類の機能

5.1.4.1 Tools（ツール）

Claude Code が呼び出せる関数。ファイル読み書き、API 呼び出しなど。

5.1.4.2 Resources（リソース）

Claude Code がコンテキストとして読み込めるデータ。ドキュメント、DB レコードなど。

5.1.4.3 Prompts（プロンプト）

再利用可能なプロンプトテンプレート。

5.1.5 MCP vs 組み込みツール

| 観点 | 組み込みツール（Read/Edit/Bash 等） | MCP ツール |
|------|---------------------------|--------------------------------|
| 目的 | ローカルファイル・シェル操作 | 外部サービス連携 |
| 設定 | 不要 | <code>settings.json</code> に記述 |
| 追加方法 | できない | MCP サーバーをインストール |
| 認証 | 不要 | API キー等が必要 |

5.1.6 公式 MCP サーバー一覧

modelcontextprotocol/servers (<https://github.com/modelcontextprotocol/servers>)

で多数の公式サーバーが公開されています：

| サーバー | 用途 |
|---|---------------|
| @modelcontextprotocol/server-filesystem | ファイルシステム操作 |
| @modelcontextprotocol/server-github | GitHub 操作 |
| @modelcontextprotocol/server-sqlite | SQLite DB 操作 |
| @modelcontextprotocol/server-brave-search | Web 検索 |
| @modelcontextprotocol/server-slack | Slack 操作 |
| @modelcontextprotocol/server-postgres | PostgreSQL 操作 |

5.1.7 次のステップ

→ 5-2: MCP サーバー設定 (02-mcp-servers.md) に進む

5.2 5-2: MCP サーバー設定

学習時間: 20分 | 難易度: ★★ ★

5.2.1 設定方法

MCP サーバーは `settings.json` の `mcpServers` セクションに記述します。

5.2.2 基本的な設定例

5.2.2.1 ファイルシステムサーバー

```
{
  "mcpServers": {
    "filesystem": {
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-filesystem",
        "/Users/me/projects"
      ]
    }
  }
}
```

5.2.2.2 GitHub サーバー

```
{
  "mcpServers": {
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": {
        "GITHUB_PERSONAL_ACCESS_TOKEN": "ghp_xxxxxxxxxxxxx"
      }
    }
  }
}
```

5.2.2.3 SQLite サーバー

```
{
  "mcpServers": {
    "sqlite": {
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-sqlite",
        "/path/to/database.db"
      ]
    }
  }
}
```

5.2.3 複数サーバーの同時使用

```
{
  "mcpServers": {
    "filesystem": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-filesystem",
"/Users/me/projects"]
    },
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": { "GITHUB_PERSONAL_ACCESS_TOKEN": "ghp_..." }
    },
    "slack": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-slack"],
      "env": { "SLACK_BOT_TOKEN": "xoxb-..." }
    }
  }
}
```

5.2.4 設定項目の説明

| フィールド | 説明 |
|------------------------|---|
| <code>command</code> | MCP サーバーを起動するコマンド |
| <code>args</code> | コマンドの引数（配列） |
| <code>env</code> | 環境変数（API キー等） |
| <code>transport</code> | 通信方式（ <code>stdio</code> （デフォルト） / <code>sse</code> ） |

5.2.5 SSE（Server-Sent Events）トランスポート

HTTP ベースで動作するリモート MCP サーバーに接続する場合：

```
{
  "mcpServers": {
    "remote-server": {
      "transport": "sse",
      "url": "https://my-mcp-server.example.com/sse",
      "headers": {
        "Authorization": "Bearer my-token"
      }
    }
  }
}
```

5.2.6 MCP サーバーの動作確認

claude

> 利用可能な MCP ツールを一覧表示して

または：

> GitHub の自分のリポジトリを一覧表示して

GitHub MCP サーバーが正しく設定されていれば、リポジトリ一覧が返ってきます。

5.2.7 実践例: GitHub と連携する

- > my-project の未解決 Issue を優先度順に並べて、
- > 今週取り組むべきものをピックアップして

> Issue #42 に「調査中」のラベルを付けて、担当者を自分に設定して

5.2.8 セキュリティの注意

- API キーは `settings.local.json` に書く（git 管理外）
- 必要最小限の権限（スコープ）を付与する
- 本番 DB への MCP 接続は慎重に

5.2.9 次のステップ

→ 5-3: エージェント SDK (03-agent-sdk.md) に進む

5.3 5-3: エージェント SDK

学習時間: 20分 | 難易度: ★★ ★

5.3.1 Claude Agent SDK とは

Claude Agent SDK は、Claude Code のエージェント機能をプログラムから呼び出すための SDK です。カスタムエージェントの構築や、Claude Code の機能を自前のアプリに組み込みます。

5.3.2 SDK のインストール

```
npm install @anthropic-ai/claude-code
```


5.3.3 基本的な使い方

5.3.3.1 シンプルなエージェント実行

```
import { query } from "@anthropic-ai/claude-code";

const result = await query({
  prompt: "src/utils/helpers.ts を読んで、改善点を教えて",
  options: {
    maxTurns: 5,
  },
});

console.log(result.result);
```

5.3.3.2 ツールの制限

セキュリティのため、使用するツールを制限できます：

```
const result = await query({
  prompt: "プロジェクトのファイル構造を分析して",
  options: {
    allowedTools: ["Read", "Glob", "Grep"],
    maxTurns: 10,
  },
});
```

5.3.3.3 ストリーミング

結果をリアルタイムでストリーミングできます：

```
import { query, type SDKMessage } from "@anthropic-ai/claude-code";

for await (const message of query({
  prompt: "大きなリファクタリングをして",
  options: { maxTurns: 20 },
})) {
  if (message.type === "assistant") {
    process.stdout.write(message.message.content);
  }
}
```

5.3.4 メッセージタイプ

SDK が返すメッセージの種類：

| タイプ | 内容 |
|-------------|---------------|
| assistant | Claude のレスポンス |
| tool_use | ツール呼び出し |
| tool_result | ツールの実行結果 |
| result | 最終結果（成功/失敗） |
| error | エラー情報 |

5.3.5 オプション一覧

```
interface QueryOptions {  
  maxTurns?: number;           // 最大ターン数（デフォルト: 10）  
  allowedTools?: string[];     // 許可するツール  
  disallowedTools?: string[];  // 禁止するツール  
  systemPrompt?: string;      // システムプロンプトの追加  
  cwd?: string;               // 作業ディレクトリ  
  model?: string;             // 使用モデル  
}
```

5.3.6 実践例: 自動コードレビュースクリプト

```
import { query } from "@anthropic-ai/claude-code";  
import { execSync } from "child_process";  
  
async function reviewPR(prNumber: number) {  
  const diff = execSync(`git diff main...HEAD`).toString();  
  
  const result = await query({  
    prompt: `以下の diff をレビューして、問題点を JSON 形式で返して：  
\n\n${diff}`,  
    options: {  
      allowedTools: ["Read", "Glob", "Grep"],  
      maxTurns: 5,  
    },  
  });  
  
  return JSON.parse(result.result);  
}
```

5.3.7 実践例: CI/CD での使用

```
# .github/workflows/review.yml
- name: AI Code Review
  run: |
    node scripts/ai-review.js
  env:
    ANTHROPIC_API_KEY: ${ secrets.ANTHROPIC_API_KEY }
```

```
// scripts/ai-review.js
const { query } = require("@anthropic-ai/claude-code");

async function main() {
  const result = await query({
    prompt: "このブランチの変更をセキュリティの観点でレビューして",
    options: { maxTurns: 10 },
  });

  if (result.result.includes("HIGH_SEVERITY")) {
    process.exit(1);
  }
}

main().catch(console.error);
```

5.3.8 次のステップ

→ 5-4: マルチエージェント (04-multi-agent.md) に進む

5.4 5-4: マルチエージェント

学習時間: 20分 | 難易度: ★★ ★

5.4.1 マルチエージェントとは

複数の Claude エージェントを並列または順次に行き、複雑なタスクを効率的に処理する手法です。Claude Code はメインエージェントからサブエージェントを起動できます。

5.4.2 なぜマルチエージェントが必要か

| 問題 | マルチエージェントによる解決 |
|------------------------|----------------------|
| コンテキスト
ウィンドウの
上限 | タスクを分割して各エージェントに割り当て |
| 複数の独立
した調査 | 並列実行で高速化 |
| 専門的なレ
ビュー | 役割特化したエージェントを使い分け |
| 大規模コー
ドベース | 並列に異なる部分を分析 |

5.4.3 2つの利用方法

マルチエージェントには **用途が異なる2つの呼び出し方**があります：

| 方法 | 誰が呼び出すか | コードは必要か | 用途 |
|----------------|--------------|--------------------|---------------------|
| チャット
(自動) | Claude が自動判断 | 不要 | Claude Code を使う日常作業 |
| Agent SDK (直接) | 自分のスクリプト | 必要 (TypeScript/JS) | 独自アプリに組み込む場合 |

5.4.4 Claude Code での使い方 (チャット・コード不要)

チャットで指示するだけで、Claude が自動的にサブエージェントを起動します。ユーザー側でコードを書く必要はありません。

- > src/components/ 以下の全コンポーネントをセキュリティレビューして。
- > 並列で複数のエージェントを使って高速化して

Claude Code は指示を解釈し、内部で Agent ツールを呼び出して並列実行します。

5.4.5 Agent SDK でのマルチエージェント (独自アプリに組み込む場合)

前提: 独自のスクリプトやアプリから Claude Code を制御したい場合に使います。日常的なチャット作業には不要です。

```
import { query } from "@anthropic-ai/claude-code";

async function parallelReview(files: string[]) {
  // 複数ファイルを並列でレビュー
  const reviews = await Promise.all(
    files.map((file) =>
      query({
        prompt: `${file} をセキュリティレビューして JSON で返して`,
        options: {
          allowedTools: ["Read"],
          maxTurns: 3,
        },
      })
    )
  );

  // query() は { result: string, ... } を返す。result がエージェントの最終出力テキスト
  return reviews.map((r) => JSON.parse(r.result));
}

// 使用例：3ファイルを並列レビューし、JSON 配列として結果を受け取る
const results = await parallelReview([
  "src/auth/login.ts",
  "src/auth/session.ts",
  "src/api/users.ts",
]);
// results[0] → login.ts のレビュー結果オブジェクト
// results[1] → session.ts のレビュー結果オブジェクト
// results[2] → users.ts のレビュー結果オブジェクト
```

5.4.6 オーケストレーターとサブエージェントのパターン

```
// オーケストレーター
async function orchestrate() {
  // Step 1: 調査エージェント（並列）
  const [authIssues, apiIssues, dbIssues] = await Promise.all([
    query({ prompt: "認証コードの問題を調査して", ... }),
    query({ prompt: "API エンドポイントの問題を調査して", ... }),
    query({ prompt: "DB クエリの問題を調査して", ... }),
  ]);

  // Step 2: 修正エージェント（調査結果を基に）
  const fixPrompt = `
    以下の問題を修正して：
    - 認証: ${authIssues.result}
    - API: ${apiIssues.result}
    - DB: ${dbIssues.result}
  `;

  const fix = await query({
    prompt: fixPrompt,
    options: { maxTurns: 20 },
  });

  return fix;
}
```

5.4.7 特化型エージェントパターン

Claude Code では `subagent_type` で専門エージェントを選べます：

| エージェント | 用途 |
|---------|--------------|
| Explore | コードベースの調査・検索 |
| Plan | 実装計画の立案 |
| claude | 汎用エージェント |

＞ Explore エージェントを使って、src/ 全体の認証関連コードを調査して

5.4.8 ワークツリーによる並列開発

Git ワークツリーを使うと、同じリポジトリの別のブランチで複数のエージェントが並列作業できます：

- ＞ 2つのエージェントを使って、
- ＞ エージェント1: パフォーマンス改善
- ＞ エージェント2: セキュリティ修正
- ＞ を並列で行って、最後にマージして

5.4.9 注意事項

- サブエージェントはそれぞれコストが発生する
- 並列実行はコンテキストを共有しない
- 結果の統合はオーケストレーターが責任を持つ
- デッドロック（相互待ち）に注意する

5.4.10 次のステップ

→ 5-5: カスタムエージェント実装 (05-custom-agents.md) に進む

5.5 5-5: カスタムエージェント実装

学習時間: 30分 | 難易度: ★★ ★

5.5.1 カスタムエージェントとは

特定のタスクに特化したエージェントを定義し、Claude Code のエコシステムに追加できます。Claude Code 自体が持つエージェントタイプの仕組みを利用します。

5.5.2 カスタムエージェントの設計原則

1. **単一責務**: 1つのエージェントは1つのことをうまくやる
2. **ツール制限**: 必要最小限のツールだけ許可する
3. **明確なインターフェース**: 入力と出力の形式を明確にする
4. **エラーハンドリング**: 失敗ケースを想定した設計

5.5.3 実装例: PR レビューエージェント

```
import { query } from "@anthropic-ai/claude-code";
```

```
interface PRReviewResult {  
  score: number;  
  issues: Array<{  
    severity: "critical" | "major" | "minor";  
    file: string;  
    line: number;  
    message: string;  
  }>;  
  approved: boolean;  
  summary: string;  
}
```

```
async function prReviewAgent(prDiff: string):
```

```
Promise<PRReviewResult> {  
  const result = await query({  
    prompt: `
```

以下の PR diff を厳密にレビューして、JSON で結果を返して。

レビュー観点

1. バグ・ロジックエラー (critical)
2. セキュリティ問題 (critical)
3. パフォーマンス問題 (major)
4. コード品質 (minor)

出力形式 (必ずこの JSON のみを返すこと)

```
{  
  "score": 0-100,  
  "issues": [  
    {
```

```

        "severity": "critical|major|minor",
        "file": "ファイルパス",
        "line": "行番号",
        "message": "問題の説明"
    }
],
"approved": true/false,
"summary": "1~2文のサマリー"
}

## Diff
${prDiff}
`,
options: {
    allowedTools: ["Read", "Grep"],
    maxTurns: 5,
    model: "claude-opus-4-8",
},
});

const jsonMatch = result.result.match(/{\[\s\S]*\}/);
if (!jsonMatch) throw new Error("Invalid response format");

return JSON.parse(jsonMatch[0]) as PRReviewResult;
}

```

5.5.4 実装例: ドキュメント生成エージェント

```
async function docGeneratorAgent(filePath: string):  
Promise<string> {  
  const result = await query({  
    prompt: `  
${filePath} を読んで、以下の形式で JSDoc コメントを生成して。  
コメントのみを返し、コード自体は変更しないこと。
```

形式:

```
/**  
 * 関数の説明  
 * @param {型} 引数名 - 説明  
 * @returns {型} 戻り値の説明  
 * @example  
 * // 使用例  
 */  
` ,  
  options: {  
    allowedTools: ["Read"],  
    maxTurns: 3,  
  },  
});  
  
  return result.result;  
}
```

5.5.5 エージェントのパイプライン化

複数のエージェントをパイプラインとして繋げます：

```

async function qualityPipeline(files: string[]) {
  for (const file of files) {
    // Step 1: レビュー
    const review = await prReviewAgent(await getFileDiff(file));

    if (review.issues.some((i) => i.severity === "critical")) {
      // Step 2: 自動修正 (critical のみ)
      await query({
        prompt: `${file} の以下の問題を修正して：
${JSON.stringify(review.issues.filter((i) => i.severity ===
"critical"))}`,
        options: { maxTurns: 10 },
      });

      // Step 3: 修正後に再レビュー
      const reReview = await prReviewAgent(await
getFileDiff(file));
      console.log(`${file}: ${reReview.approved ? "✅" :
"❌"}`);
    }
  }
}

```

5.5.6 GitHub Actions でのカスタムエージェント

```
name: AI Quality Gate
on: [pull_request]

jobs:
  ai-review:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Run AI Review Agent
        run: node scripts/pr-review-agent.js
        env:
          ANTHROPIC_API_KEY: ${ secrets.ANTHROPIC_API_KEY }
          PR_DIFF: ${ github.event.pull_request.diff_url }
```

5.5.7 次のステップ

→ 6-1: CI/CD 統合 (../06-advanced/01-ci-cd-integration.md) に進む

6. 発展・応用

6.1 6-1: CI/CD 統合

学習時間: 20分 | 難易度: ★★ ★

6.1.1 概要

Claude Code を CI/CD パイプラインに組み込むことで、コードレビューや品質チェックを自動化できます。

6.1.2 GitHub Actions での基本的な使い方

6.1.2.1 PR 自動コードレビュー

```
# .github/workflows/ai-review.yml
name: AI Code Review

on:
  pull_request:
    types: [opened, synchronize]

jobs:
  review:
    runs-on: ubuntu-latest
    permissions:
      pull-requests: write
      contents: read

    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: "20"

      - name: Install Claude Code
        run: npm install -g @anthropic-ai/claude-code

      - name: Run AI Review
        run: |
```

```
git diff origin/main...HEAD > /tmp/pr.diff  
node scripts/review.js
```

env:

```
ANTHROPIC_API_KEY: ${ secrets.ANTHROPIC_API_KEY }  
GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }
```

6.1.2.2 レビュースクリプト

```
// scripts/review.js
const { query } = require("@anthropic-ai/claude-code");
const { Octokit } = require("@octokit/rest");
const fs = require("fs");

async function main() {
  const diff = fs.readFileSync("/tmp/pr.diff", "utf8");

  const result = await query({
    prompt: `
以下の PR diff をレビューして。
重大な問題（バグ・セキュリティ・パフォーマンス）のみを報告して。
JSON 形式で返して。

${diff}
`,
    options: {
      allowedTools: ["Read", "Grep"],
      maxTurns: 5,
    },
  });

  const issues = JSON.parse(result.result);

  if (issues.critical_count > 0) {
    console.error("Critical issues found");
    process.exit(1);
  }

  console.log("Review passed");
}
```

```
main().catch(console.error);
```

6.1.3 GitLab CI での使用

6.1.3.1 GitLab CI の基本概念

GitLab CI は `.gitlab-ci.yml` という1つのファイルでパイプライン全体を定義します。

| 用語 | 意味 | GitHub Actions との対応 |
|--------------------|-------------------------------------|---------------------|
| Pipeline | CI 全体の実行単位 | Workflow |
| Stage | パイプラインの段階（例: build → test → review） | - |
| Job | 各ステージ内の個別タスク | Job |
| Runner | ジョブを実行するサーバー | Runner |
| Merge Request (MR) | コードレビューの単位 | Pull Request (PR) |

6.1.3.2 API キーの設定方法

GitLab プロジェクトの **Settings** → **CI/CD** → **Variables** で以下を登録します：

| 変数名 | 値 | オプション |
|--------------------------------|-------------------------|------------------|
| <code>ANTHROPIC_API_KEY</code> | <code>sk-ant-...</code> | Masked（ログに表示しない） |

6.1.3.3 基本設定

```
# .gitlab-ci.yml
ai-review:
  stage: review
  image: node:20
  script:
    - npm install -g @anthropic-ai/claude-code
    - git diff origin/main...HEAD > /tmp/pr.diff
    - node scripts/review.js
  variables:
    ANTHROPIC_API_KEY: $ANTHROPIC_API_KEY
  only:
    - merge_requests
```

6.1.3.4 設定の各項目の意味

| キー | 説明 |
|----------------------|---|
| stage: review | stages: で定義したステージの1つに配置する |
| image: node:20 | ジョブを実行する Docker イメージ (Node.js 20) |
| script: | 実行するシェルコマンドのリスト (上から順に実行) |
| variables: | ジョブ内で使う環境変数 (\$変数名 で GitLab CI 変数を参照) |
| only: merge_requests | マージリクエストが作成・更新されたときだけ実行 |

6.1.3.5 複数ステージを使った実践的な構成

```
# .gitlab-ci.yml (マルチステージ版)

stages:
  - build
  - test
  - review    # ← AI レビューはここ
  - deploy

ai-review:
  stage: review
  image: node:20
  script:
    - npm install -g @anthropic-ai/claude-code
    - git diff origin/main...HEAD > /tmp/pr.diff
    - node scripts/review.js
  variables:
    ANTHROPIC_API_KEY: $ANTHROPIC_API_KEY
  artifacts:
    paths:
      - review-report.json    # レポートを次のジョブに引き継ぐ
    expire_in: 1 week
  only:
    - merge_requests
```

ポイント: `artifacts` を使うとレビュー結果ファイルを保存・ダウンロードできます。

6.1.3.6 GitLab と GitHub Actions の比較

| 項目 | GitLab CI | GitHub Actions |
|----------------|---|--|
| 設定ファイル | <code>.gitlab-ci.yml</code> (1ファイル) | <code>.github/workflows/*.yml</code> (複数可) |
| トリガー | <code>only:</code>
<code>merge_requests</code> | <code>on: pull_request:</code> |
| シークレット | Settings → CI/CD → Variables | Settings → Secrets |
| セルフホスト
実行環境 | GitLab Runner | Self-hosted Runner |

6.1.4 品質ゲートとしての使用

コードレビューが通らなければマージをブロックするパターン：

```
# GitHub Actions
- name: AI Security Gate
  run: |
    node scripts/security-gate.js
  env:
    ANTHROPIC_API_KEY: ${ secrets.ANTHROPIC_API_KEY }

# セキュリティ問題があれば exit 1 でジョブを失敗させる
```

```
// scripts/security-gate.js
const { query } = require("@anthropic-ai/claude-code");

async function securityGate() {
  const result = await query({
    prompt: "このブランチの変更をセキュリティレビューして。HIGH 以上の問題があれば FAIL と出力して",
    options: {
      allowedTools: ["Read", "Grep", "Glob"],
      maxTurns: 10,
    },
  });

  if (result.result.includes("FAIL")) {
    console.error("Security gate failed");
    process.exit(1);
  }

  console.log("Security gate passed");
}
```

6.1.5 コスト管理

CI/CD で使用する場合はコストに注意：

| 戦略 | 説明 |
|--------------|--|
| maxTurns を制限 | <code>maxTurns: 3~5</code> で制限 |
| 軽量モデルを使う | <code>claude-haiku-4-5-20251001</code> を CI/CD に |
| 変更ファイルのみ対象 | diff ファイルのみを読む |
| キャッシュを活用 | 変更がないファイルはスキップ |

6.1.6 次のステップ

→ 6-2: チームワークフロー (02-team-workflow.md) に進む

6.2 6-2: チームワークフロー

学習時間: 15分 | 難易度: ★★ ★

6.2.1 チームでの Claude Code 活用

チームで Claude Code を使う際は、個人の設定とチーム共有の設定を適切に分離することが重要です。

6.2.2 設定の分離

```
my-project/
├── .claude/
│   ├── settings.json           # チームで共有 (git 管理)
│   ├── settings.local.json     # 個人設定 (.gitignore に追加)
│   └── skills/                 # チーム共有スキル (git 管理)
│       ├── deploy-check/
│       └── release-notes/
├── CLAUDE.md                   # プロジェクト指示 (git 管理)
└── .gitignore
    # .claude/settings.local.json を追加
```

6.2.2.1 .gitignore に追加

```
# Claude Code 個人設定
.claude/settings.local.json
```

6.2.3 チーム共有の settings.json

```
{
  "permissions": {
    "allow": [
      "Bash(git status)",
      "Bash(git diff *)",
      "Bash(git log *)",
      "Bash(git add *)",
      "Bash(git commit *)",
      "Bash(npm run *)",
      "Bash(npm test)"
    ],
    "deny": [
      "Bash(git push --force *)",
      "Bash(rm -rf *)"
    ]
  },
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "npm run format 2>/dev/null || true"
          }
        ]
      }
    ]
  }
}
```

6.2.4 効果的な CLAUDE.md の書き方（チーム向け）

プロジェクト名

チームルール

- PR は必ず 1 機能ずつに分割する
- マージ前に `/code-review` を実行すること
- ブランチ名は `feature/`, `fix/`, `chore/` プレフィックスを使う

禁止事項

- main ブランチへの直接 push
- console.log のコミット
- `any` 型の使用

コードオーナー

- src/auth/ → @auth-team
- src/payments/ → @payments-team

6.2.5 チーム共有スキルの管理

プロジェクトスキルをリポジトリに含めることでチーム全員が同じスキルを使えます：

```
.claude/skills/  
├─ release-notes/  
|   └─ SKILL.md    # リリースノート自動生成  
├─ db-migration/  
|   └─ SKILL.md    # DB マイグレーション安全チェック  
└─ api-review/  
    └─ SKILL.md    # API 設計レビュー
```

6.2.6 オンボーディングへの活用

新メンバーのオンボーディングに CLAUDE.md を活用：

新メンバーへ

このプロジェクトに初めて参加する場合は、以下を実行してください：

```
``bash
npm install
cp .env.example .env.local
npm run db:setup
npm run dev
``
```

Claude Code で `/init` を実行すると、プロジェクト全体の概要を説明します。

6.2.7 レビュー文化への組み込み

6.2.7.1 PR テンプレートに Claude Code を組み込む

```
<!-- .github/pull_request_template.md -->
## チェックリスト

- [ ] `/code-review` を実行した
- [ ] `/security-review` を実行した（認証・DB 変更がある場合）
- [ ] テストを追加・更新した
- [ ] CLAUDE.md のコーディング規約を確認した
```

6.2.8 次のステップ

→ 6-3: パフォーマンス Tips (03-performance-tips.md) に進む

6.3 6-3: パフォーマンス Tips

学習時間: 15分 | 難易度: ★★ ★

6.3.1 レスpons速度の最適化

6.3.1.1 Fast モードを使う

```
/fast
```

Claude Opus の高速出力モードに切り替えます。複雑な推論が不要な作業に適しています。

6.3.1.2 用途に合ったモデルを選ぶ

| タスク | 推奨モデル |
|---------------|---------------------------|
| 複雑なアーキテクチャ設計 | claude-opus-4-8 |
| 通常の開発作業 | claude-sonnet-4-6 |
| 単純な編集・整形 | claude-haiku-4-5-20251001 |
| CI/CD の自動チェック | claude-haiku-4-5-20251001 |

6.3.2 コンテキストの最適化

6.3.2.1 不要なファイルを読ませない

```
# 悪い例
> プロジェクト全体を読んで、Button コンポーネントを修正して

# 良い例
> src/components/Button.tsx を読んで修正して
```

6.3.2.2 CLAUDE.md を簡潔に保つ

CLAUDE.md は毎回読み込まれます。必要最小限の情報だけ書きましょう：

- 技術スタック（5～10行）
- よく使うコマンド（5～10個）
- 重要な規約（5～10行）

6.3.2.3 /clear を適切に使う

長い会話はパフォーマンスが低下します。タスクが完了したら `/clear` でリセットしましょう。

6.3.3 プロンプトの最適化

6.3.3.1 具体的に指示する

- # 悪い例（曖昧）
 - > このコードを改善して

- # 良い例（具体的）
 - > `UserList` コンポーネントの `useEffect` を最適化して。
 - > 現在は毎レンダリングで API 呼び出しが発生している。
 - > `userId` が変わったときだけ呼び出すよう修正して。

6.3.3.2 出力形式を指定する

- > テストカバレッジが低い関数を10個、JSON 配列で返して。
- > 各要素は { `file`, `function`, `currentCoverage` } の形式で。

6.3.4 API コストの最適化

6.3.4.1 プロンプトキャッシングの活用

長い CLAUDE.md や参照ドキュメントは自動的にキャッシュされます。繰り返し参照するドキュメントは CLAUDE.md に書いておくとコストが下がります。

6.3.4.2 CI/CD での最適化

```
// CI/CD では軽量モデルを使う
const result = await query({
  prompt: "...",
  options: {
    model: "claude-haiku-4-5-20251001", // 高速・低コスト
    maxTurns: 3,                        // ターン数を制限
    allowedTools: ["Read", "Grep"],     // ツールを制限
  },
});
```

6.3.5 サブエージェントの活用

時間のかかる調査はサブエージェントに任せることでメインの会話をブロックせずに進められます：

- ＞ バックグラウンドで `src/legacy/` の依存関係を調査して。
- ＞ 終わったら教えて。それまで別の作業を続けよう。

6.3.6 よくあるボトルネック

| 症状 | 原因 | 対処法 |
|--------------|------------------|-------------------|
| レスポンスが遅い | コンテキストが大きすぎる | /clear でリセット |
| 同じ情報を何度も確認する | CLAUDE.md が不完全 | CLAUDE.md を充実させる |
| ファイルを何度も読み直す | 関連ファイルを先に読ませる | 最初に必要ファイルを指定 |
| 承認待ちが多い | permissions が未設定 | settings.json を設定 |

6.3.7 次のステップ

→ 6-4: トラブルシューティング (04-troubleshooting.md) に進む

6.4 6-4: トラブルシューティング

学習時間: 15分 | 難易度: ★★ ★

6.4.1 よくある問題と解決策

6.4.1.1 Claude Code が起動しない

症状: `claude` コマンドが見つからない

```
# 確認
which claude          # macOS/Linux
claude --version      # インストール確認
claude doctor         # 自動診断
```

再インストール（ネイティブインストーラーが推奨）：

```
# macOS/Linux
curl -fsSL https://claude.ai/install.sh | bash

# Windows PowerShell
irm https://claude.ai/install.ps1 | iex

# Homebrew (macOS)
brew install --cask claude-code

# WinGet (Windows)
winget install Anthropic.ClaudeCode
```

インストール先: `~/.local/bin/claude` (macOS/Linux) /
`%USERPROFILE%\local\bin\claude.exe` (Windows)

症状: `ANTHROPIC_API_KEY` が未設定

```
# 確認
echo $ANTHROPIC_API_KEY

# 設定
export ANTHROPIC_API_KEY="sk-ant-..."

# 永続化 (~/.bashrc または ~/.zshrc に追加)
echo 'export ANTHROPIC_API_KEY="sk-ant-..."' >> ~/.bashrc
```

6.4.1.2 ツールが承認待ちになり続ける

症状: 毎回 `Bash(git status)` の承認を求められる

解決策: `settings.json` に許可ルールを追加する

```
{
  "permissions": {
    "allow": [
      "Bash(git *)",
      "Bash(npm *)",
      "Read(**)",
      "Edit(**)"
    ]
  }
}
```

6.4.1.3 スキルが自動読み込みされない

症状: `.claude/skills/my-skill/SKILL.md` を置いたが自動起動しない

確認ポイント: 1. `description` フィールドが具体的か確認する 2. ディレクトリ名とファイル名が正しいか確認する

```
# 構造を確認
ls .claude/skills/my-skill/
# → SKILL.md が存在すること

# description を確認
head -10 .claude/skills/my-skill/SKILL.md
# → description フィールドが具体的に書かれていること
```

スラッシュコマンドで明示的に呼び出すこともできます：

```
/my-skill
```

6.4.1.4 MCP サーバーが接続できない

症状: MCP ツールが使えない

```
# MCP サーバーを手動で起動してテスト
npx -y @modelcontextprotocol/server-filesystem /tmp

# settings.json の記述を確認
cat ~/.claude/settings.json | grep -A 10 mcpServers
```

よくある原因: - `npx` のパスが通っていない - 環境変数（API キー等）が未設定
- Node.js のバージョンが古い（npm 経由インストールの場合は 18 以上が必要）

6.4.1.5 コンテキストが切れてしまう

症状: 途中で会話がリセットされる、過去の指示を忘れる

解決策: 1. 重要な指示は CLAUDE.md に書く 2. メモリに保存する（> `これを覚えておいて`） 3. `/compact` でコンテキストを圧縮しながら会話を継続する 4. 長いタスクは小さく分割して `/clear` を挟む

6.4.1.6 フックが動作しない

症状: `PostToolUse` フックが実行されない

まず試すこと:

```
/doctor
```

インストール・設定・MCP・コンテキストの状態を自動チェックしてくれる。

ログを使ったデバッグ方法:

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit",
        "hooks": [
          {
            "type": "command",
            "command": "echo 'hook triggered' >> /tmp/claude-
hooks.log"
          }
        ]
      }
    ]
  }
}
```

```
# ログを確認
tail -f /tmp/claude-hooks.log
```

6.4.1.7 API レート制限エラー

症状: `rate_limit_error` が発生する

解決策: - より軽量なモデルを使う - `maxTurns` を制限する - Anthropic
Console でレート制限を確認・引き上げ申請

6.4.1.8 Windows での文字化け

症状: 日本語が文字化けする、文字が豆腐（□）になる

VS Code / Cursor の統合ターミナルで発生する場合:

セッション内で以下を実行すると GPU レンダーが無効化される：

```
/terminal-setup
```

PowerShell のエンコーディング設定（一般的な文字化けの場合）：

```
$OutputEncoding = [System.Text.Encoding]::UTF8  
[Console]::OutputEncoding = [System.Text.Encoding]::UTF8
```

6.4.2 デバッグモード

詳細なログを出力するには：

```
# コマンドラインフラグ（推奨）  
claude --debug  
  
# 環境変数  
DEBUG=1 claude
```

セッション起動後は `/debug` コマンドでも切り替え可能。

診断レポートを確認したい場合は：

```
claude doctor
```

6.4.3 サポートリソース

| リソース | URL |
|-----------------|---|
| 公式ドキュメント | https://code.claude.com/docs/ |
| GitHub Issues | https://github.com/anthropics/claude-code/issues |
| Claude Code ガイド | <code>/help</code> コマンドで確認 |
| フィードバック送信 | <code>/feedback</code> コマンドで直接報告 |